

Инструкция к набору для моделирования Arduino Uno R3 Maximum KIT с RFID модулем



Содержание

1. Подготовка среды разработки.....	3
1.1. Установка Arduino IDE.....	3
1.2. Настройка Arduino IDE.....	4
1.3. Установка драйвера.....	7
1.4. Проверка установки.....	7
1.5. Советы по устранению проблем.....	9
1.6. Установка дополнительных библиотек для расширения функционала Arduino.....	10
2. Техника безопасности.....	14
3. Мигающий светодиод.....	16
4. Диммируемый светодиод.....	19
5. Бегущая строка из светодиодов.....	22
6. Индикатор на семисегментном дисплее.....	24
7. Управление светодиодами через регистр сдвига.....	29
8. Подключение Кнопки.....	32
9. Управление светодиодом и кнопкой.....	35
10. Чтение аналогового сигнала с потенциометра.....	37
11. Двухосевой джойстик.....	39
12. Генератор звуковой частоты.....	41
13. Игра "Быстрая дуэль"	43
14. Датчик освещенности на основе фоторезистора.....	46
15. Индикатор уровня освещенности.....	48
16. Фотосенсорный зуммер.....	50
17. Управление сервоприводом с помощью Arduino UNO.....	53
18. Управление сервоприводом с помощью потенциометра.....	55
19. Инфракрасный приемник.....	57
20. Управление шаговым двигателем.....	59
21. Измерение температуры.....	62
22. Измерение температуры и влажности с помощью DHT11.....	64
23. Считыватель RFID-карт.....	67
24. Система контроля доступа на базе RFID.....	70
25. Индикатор на ЖК-дисплее.....	73
26. Подключение ЖК-дисплея i2c.....	75
27. Часы реального времени с Arduino и RTC DS1302.....	78
28. Станция мониторинга температуры, влажности и времени.....	82
Приложение А. Состав набора.....	87

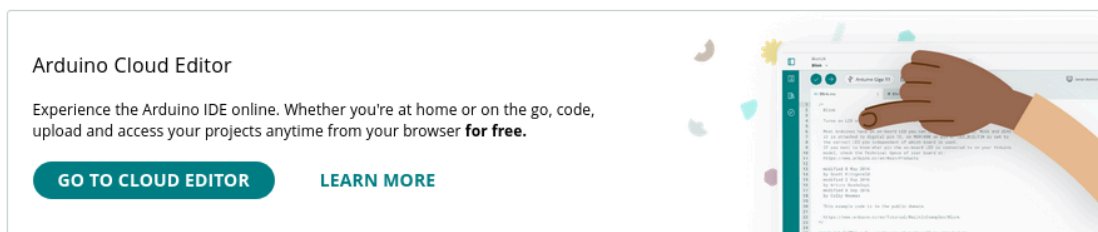
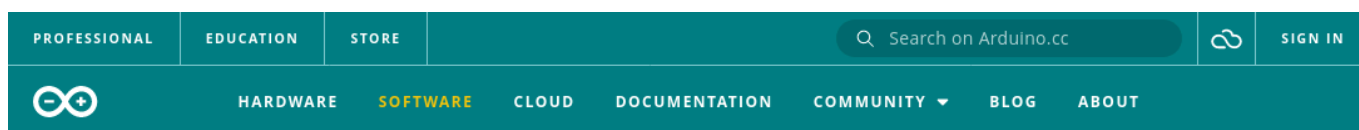
1. Подготовка среды разработки

Подготовка среды разработки является первым шагом на пути к освоению мира программирования и электроники с Arduino. Выбор правильной интегрированной среды разработки (IDE) критически важен для комфортной и продуктивной работы. Arduino IDE 2.0, с её интуитивно понятным интерфейсом, улучшенной производительностью и встроенными инструментами отладки представляет собой идеальный выбор как для новичков, так и для опытных разработчиков. В этой главе мы рассмотрим, как скачать, установить и настроить Arduino IDE 2.0, а также подготовим всё необходимое для начала работы над Вашими проектами.

В данной инструкции описаны основные моменты по установке и настройке среды разработки Arduino IDE для операционной системы Windows. Более полную инструкцию можно будет ознакомиться на официальном сайте Arduino по ссылке <https://docs.arduino.cc/software/ide-v2>.

1.1. Установка Arduino IDE

Перейдите на официальный сайт Arduino <https://www.arduino.cc/en/software>, чтобы скачать последнюю версию Arduino IDE 2.0.



Downloads



Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

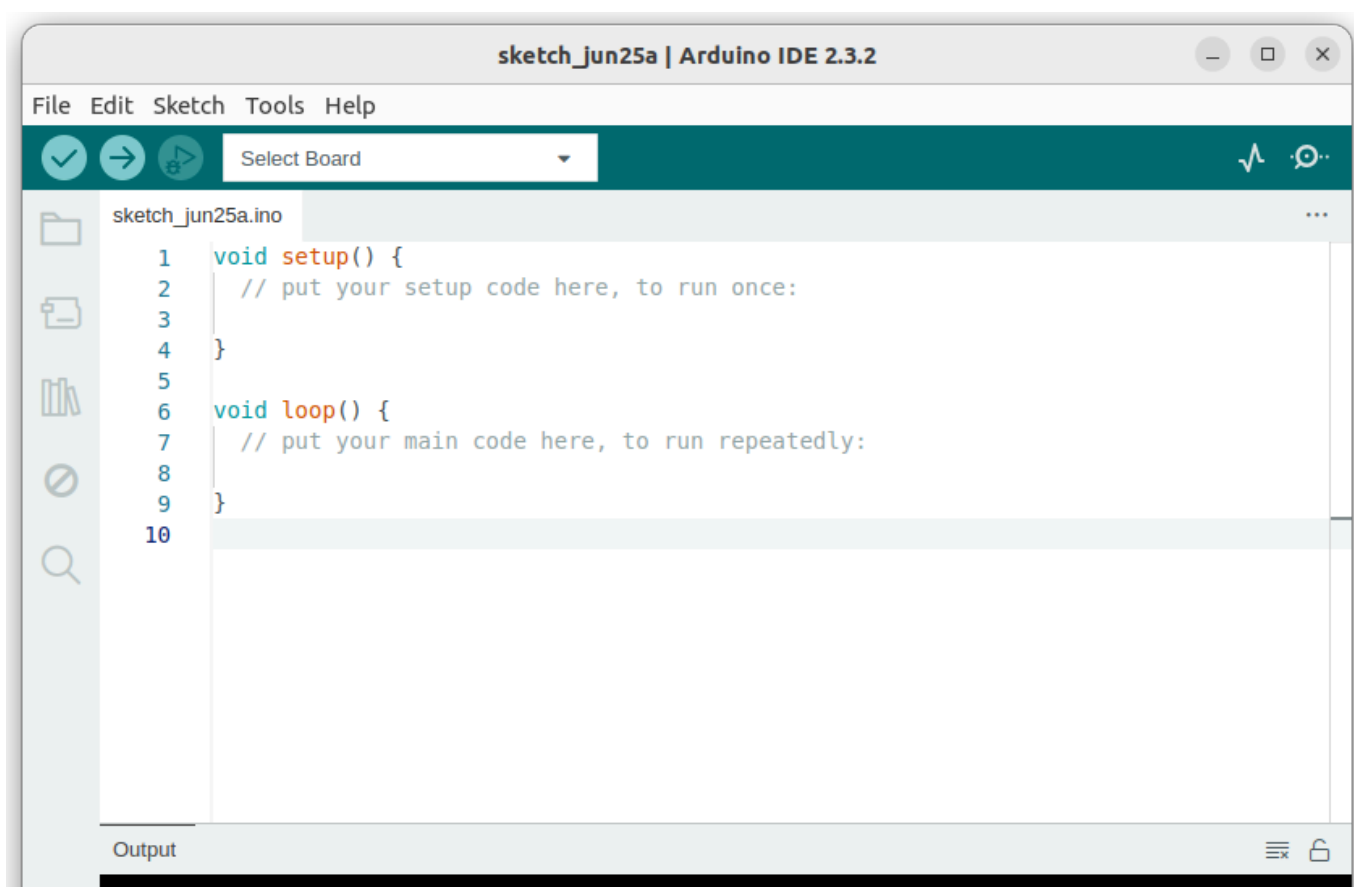
[Release Notes](#)

В разделе “DOWNLOAD OPTIONS” выберите ссылку для загрузки, той операционной системой которой пользуетесь.

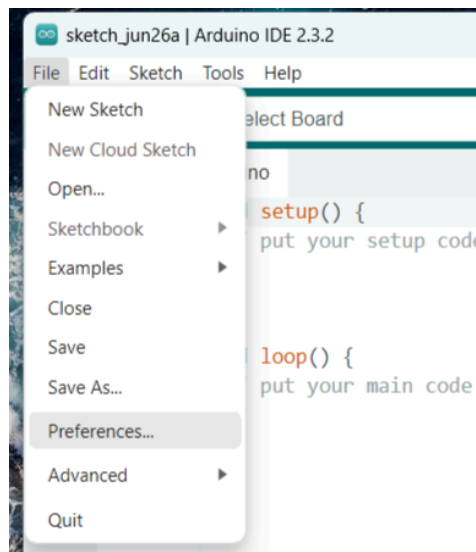
После завершения загрузки найдите скачанный файл и запустите его. Следуйте инструкциям установщика, чтобы установить Arduino IDE на Ваш компьютер. Во время установки могут появляться сообщения от системы на установку драйверов и предоставление дополнительных привилегий, требующиеся для работы Arduino IDE. На все сообщения отвечайте утвердительно. После завершения установки запустите Arduino IDE, чтобы убедиться, что все работает корректно.

1.2. Настройка Arduino IDE

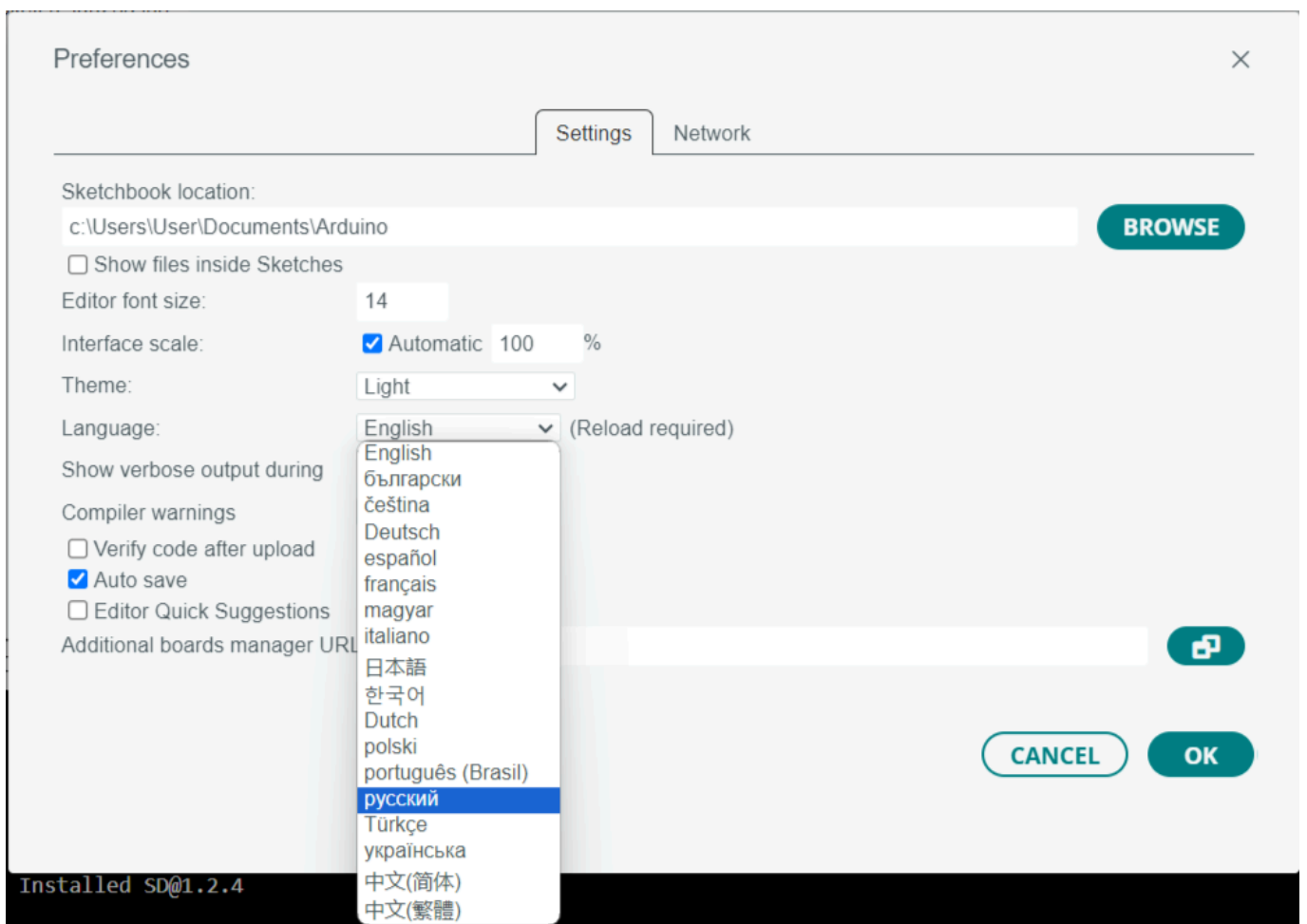
При первом запуске Arduino IDE интерфейс будет на английском языке.



Для того чтобы интерфейс перевести на русский язык, для этого в меню перейдите в раздел “File” и выберите пункт меню “Preferences”.

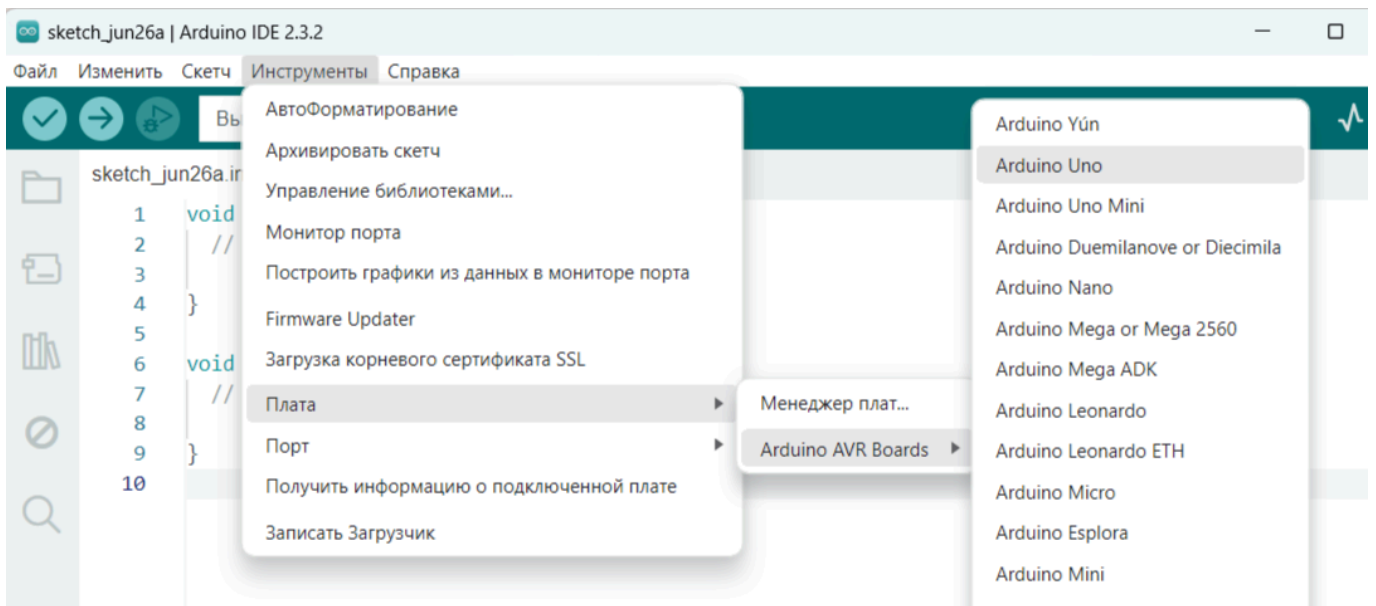


В диалоговом окне в пункте “Language” выберите язык “русский” и нажмите кнопку “OK”. теперь интерфейс на русском языке.

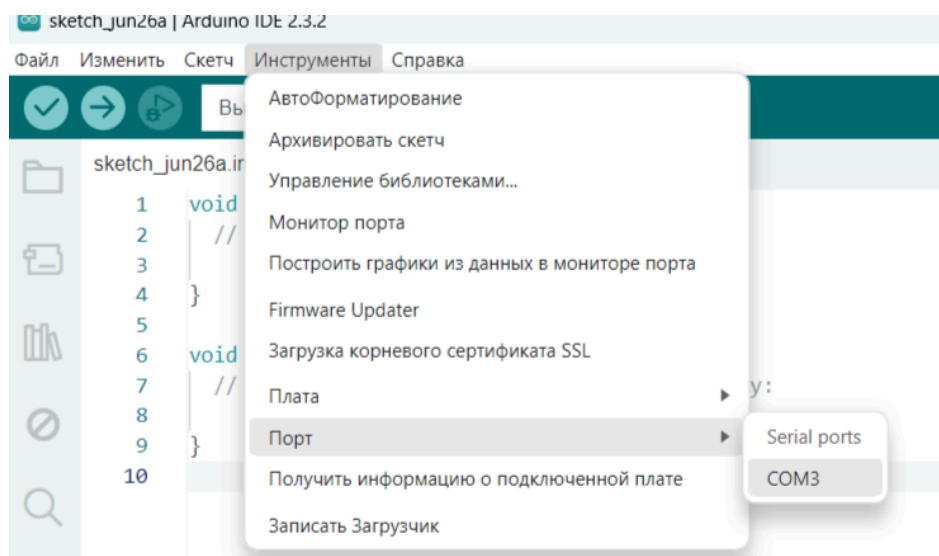


Теперь подключим нашу плату Arduino Uno R3 к компьютеру и настроим в Arduino IDE для работы с ней.

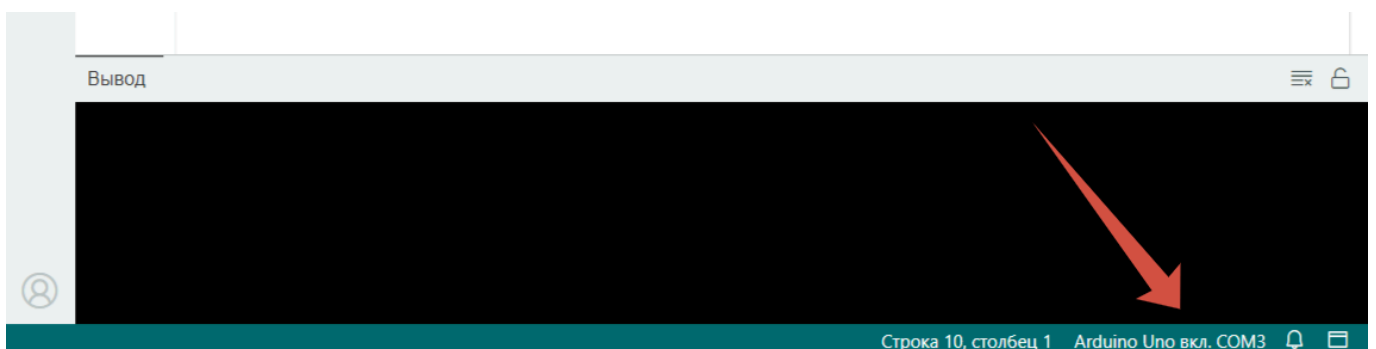
В Arduino IDE перейдем “Инструменты” -> “Плата” -> ”Arduino AVR Boards” и из списка выберем “Arduino UNO”.



Дальше выбираем порт, к которому подключена наша плата, для этого переходим “Инструменты” -> “Порт” из списка выбираем порт к которому подключена наша плата.



В нижнем правом углу можно увидеть выбранную плату и порт который будет использоваться при работе с платой.



1.3. Установка драйвера

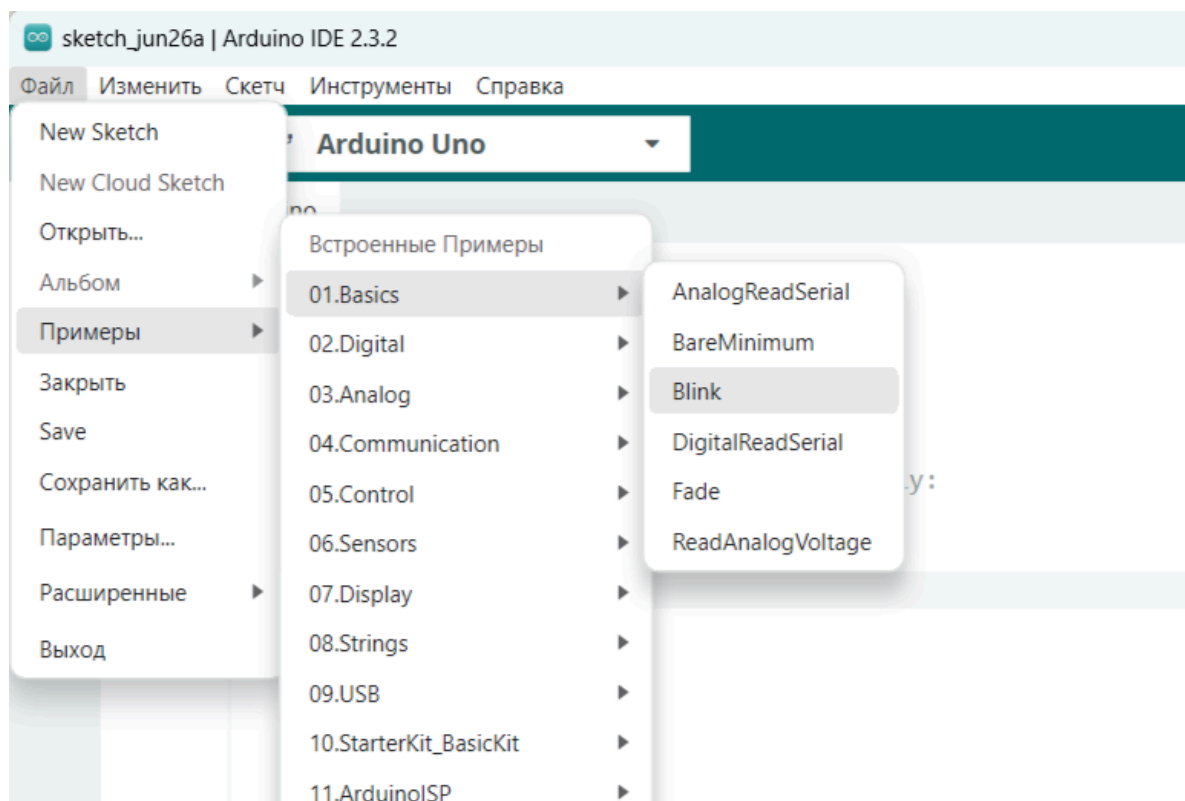
При установке Arduino IDE обычно все необходимые драйвера устанавливаются, но плата Arduino Uno R3 которая находится в комплекте, для подключения к компьютеру использует микросхему CH340, для корректной работы может потребоваться установить дополнительный драйвер. Драйвер для своей операционной системы сможете найти на официальном сайте https://www.wch-ic.com/downloads/CH341SER_EXE.html

1.4. Проверка установки

После установки и настройки Arduino IDE. Следует убедиться что все работает и можно приступать к выполнению заданий и к реализации своих проектов.

Для проверки будем использовать пример “Blink”. Данный пример использует светодиод на плате, который подключен к 13 выводу.

Загрузить пример “Blink” можно из общего списка примеров Arduino IDE.



В эскизе появится следующий код

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
```

```

}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage
level)
  delay(1000);                      // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the
voltage LOW
  delay(1000);                      // wait for a second
}

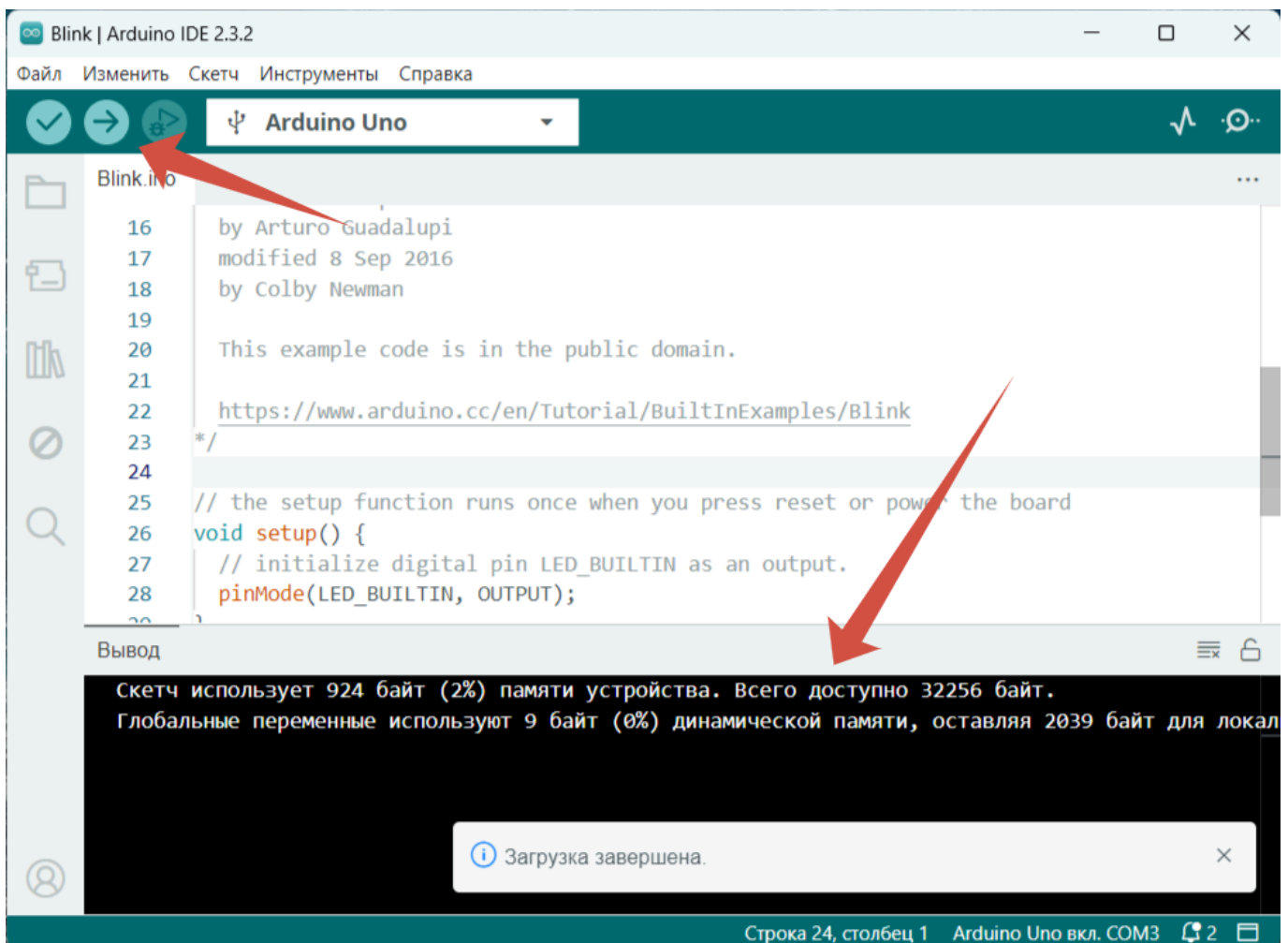
```

Принцип работы следующий:

1. Функция `setup()` вызывается один раз при запуске программы или при нажатии кнопки сброса. В ней выполняется начальная настройка:
 - Пин, к которому подключен встроенный светодиод (`LED_BUILTIN`), настраивается как выход (`OUTPUT`), что позволяет управлять светодиодом.
2. Функция `loop()` выполняется циклически бесконечно после `setup()`:
 - Сначала светодиод включается путем подачи высокого уровня напряжения (`HIGH`) на пин `LED_BUILTIN`.
 - Затем программа ожидает 1000 миллисекунд (1 секунда), в течение которых светодиод остается включенным.
 - После задержки светодиод выключается путем подачи низкого уровня напряжения (`LOW`) на тот же пин.
 - Следует еще одна задержка в 1000 миллисекунд, во время которой светодиод остается выключенным.

Этот процесс повторяется бесконечно, создавая эффект мигания светодиода с периодом в 2 секунды (1 секунда включен, 1 секунда выключен).

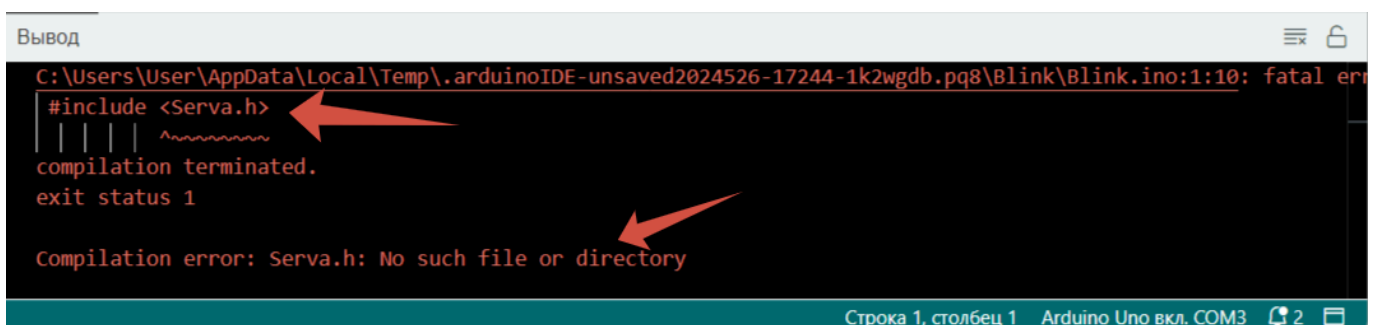
Скомпилируем и загрузим данный проект на плату Arduino.



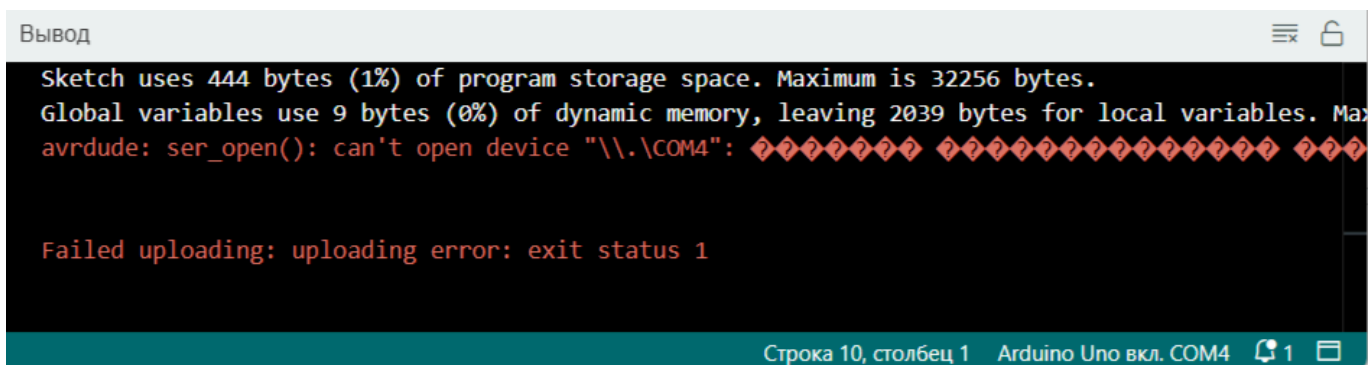
Информацию о состоянии процессе компиляции и загрузки программы на плату возможно будет увидеть в разделе “Вывод”.

1.5. Советы по устранению проблем

При выполнении компиляции и загрузки могут возникнуть ошибки, данный ошибки будут также отображаться в разделе “Вывод”. Далее приведены несколько примеров ошибок, с которыми возможно встретиться.



Подключенная библиотека не найдена. Тут нужно убедиться что библиотека данная библиотека установлена или убедиться в правильности указанной библиотеки. Данной ошибки можно избежать если подключать библиотеки через меню.



Вывод

```
Sketch uses 444 bytes (1%) of program storage space. Maximum is 32256 bytes.  
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Max  
avrdude: ser_open(): can't open device "\\.\COM4": ?????????? ?????????? ??????????  
  
Failed uploading: uploading error: exit status 1
```

Строка 10, столбец 1 Arduino Uno вкл. COM4 1 [иконка]

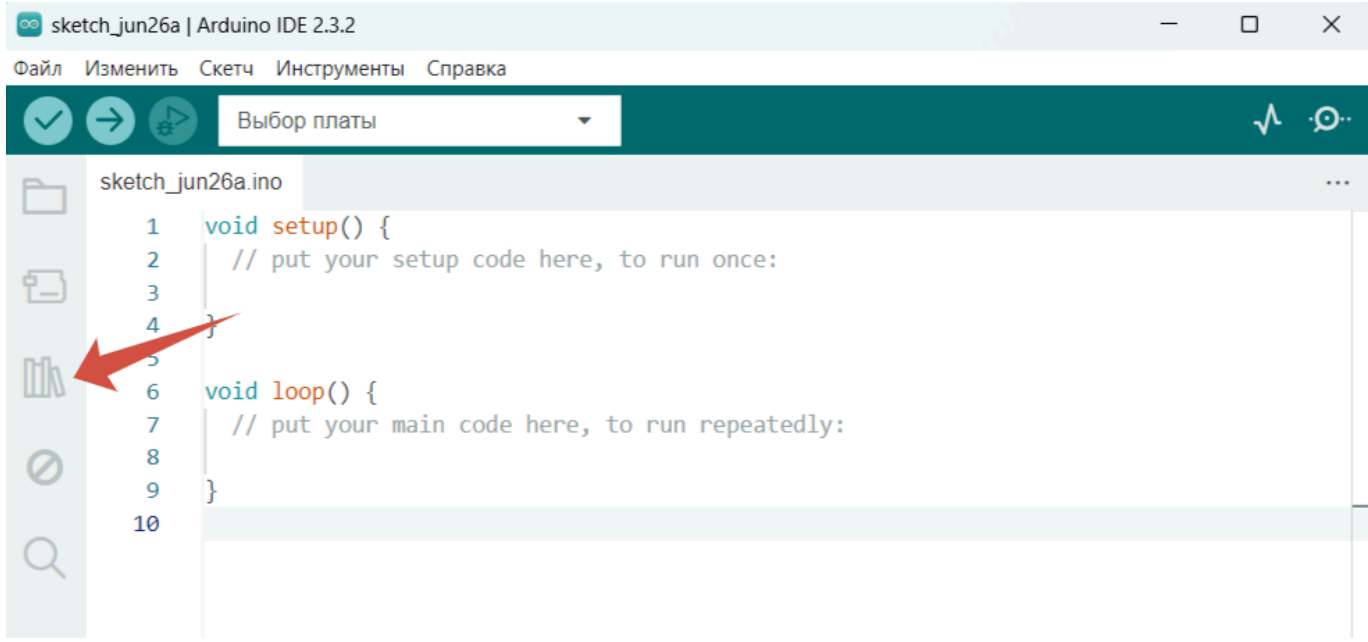
К выбранному порту, плата arduino не подключена. Проверьте подключение платы Arduino к компьютеру и проверьте правильность выбранного порта.

1.6. Установка дополнительных библиотек для расширения функционала Arduino.

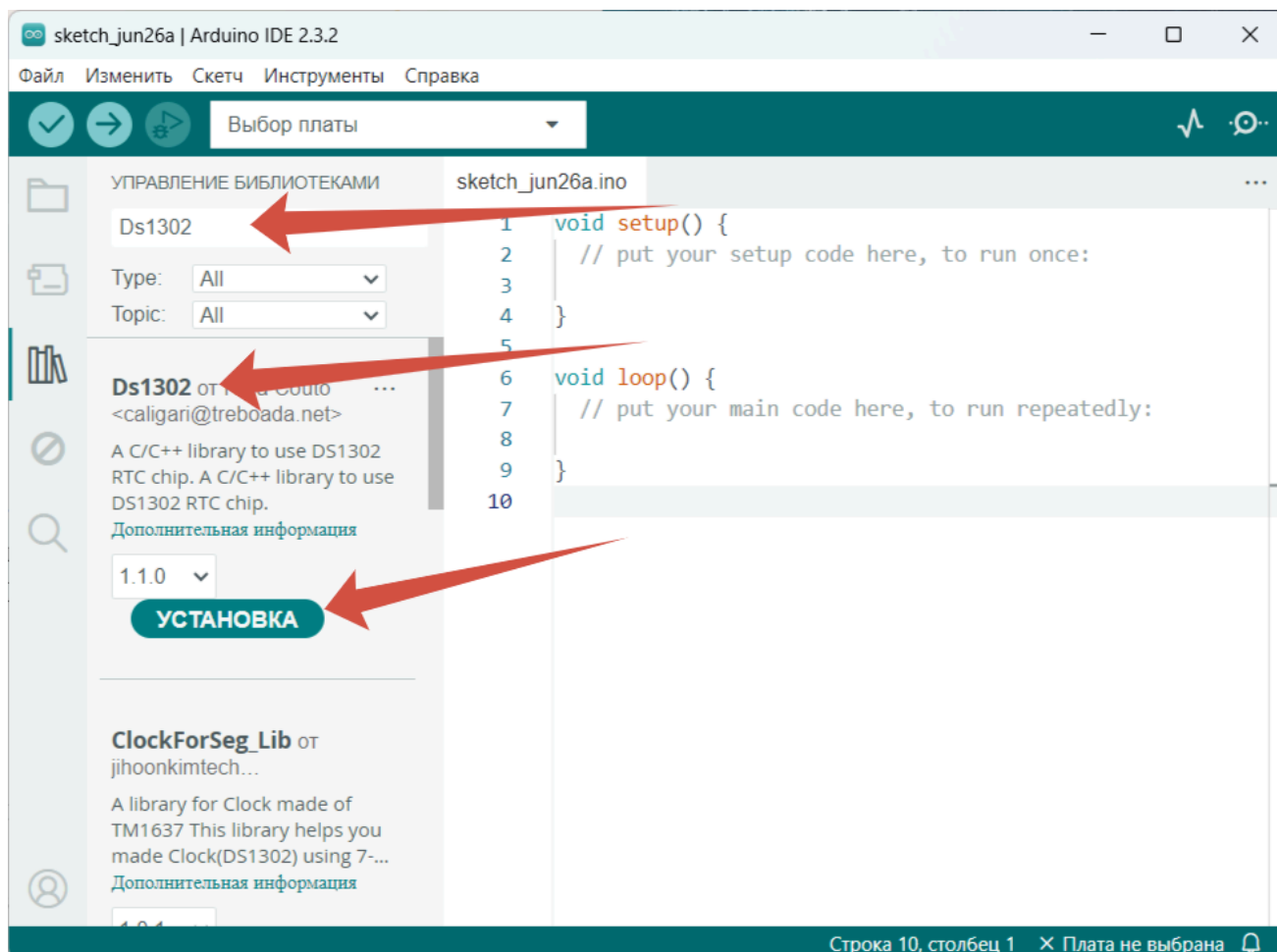
Библиотеки – это мощные инструменты, которые позволяют легко интегрировать и использовать различные модули и датчики, не погружаясь в сложности низкоуровневого программирования. Они содержат предварительно написанный код, который вы можете использовать для управления новым оборудованием или выполнения сложных задач. Независимо от того, являетесь ли вы новичком или опытным разработчиком, установка библиотек сделает процесс создания проектов более быстрым и приятным.

1.6.1 Установка библиотеки

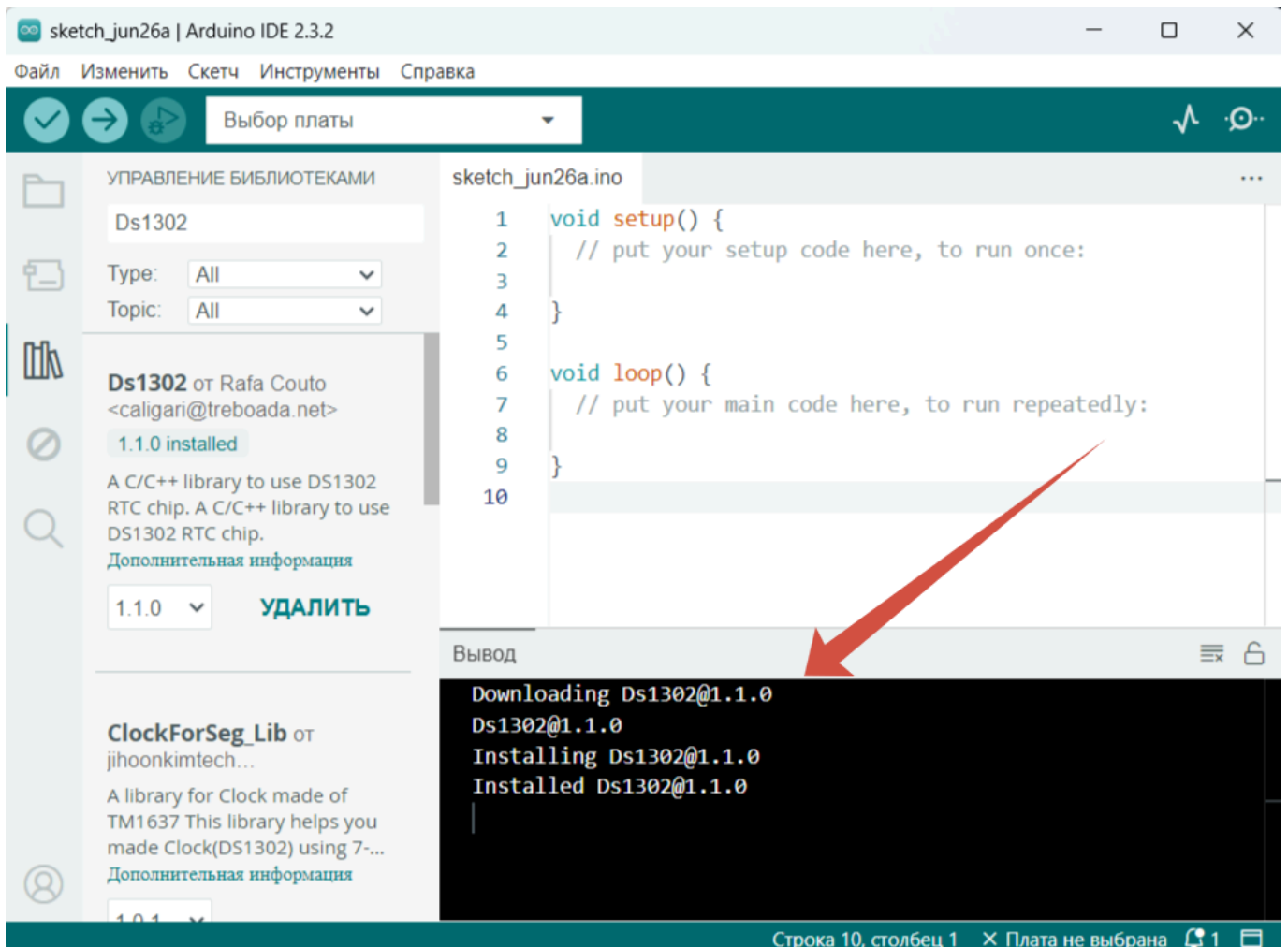
Открыв редактор, давайте посмотрим на левый столбец. Здесь мы видим пару иконок. Нажмем на значок «библиотека» .



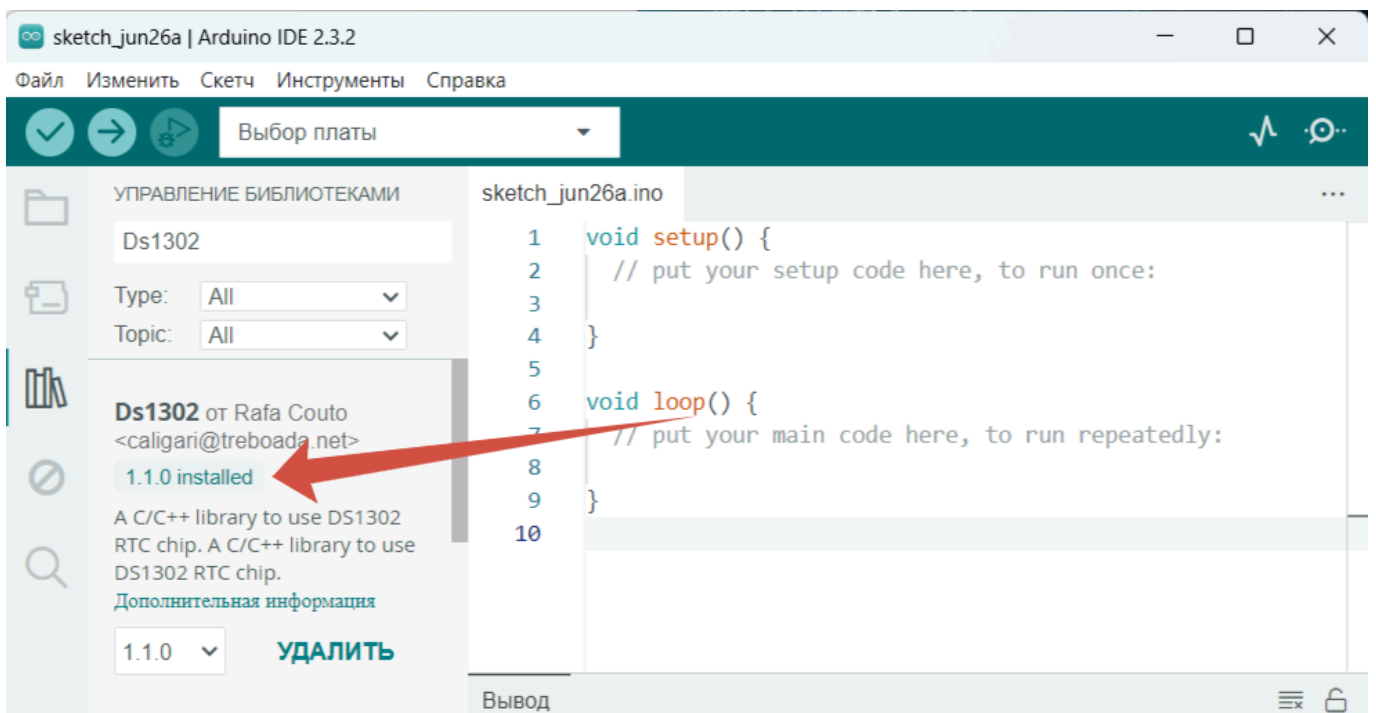
Теперь появится список всех доступных библиотек, где мы также сможем найти библиотеку, которую хотим использовать. В этом примере мы собираемся установить библиотеку Ds1302. Нажмите кнопку «УСТАНОВИТЬ», чтобы установить библиотеку.



Этот процесс не должен занять слишком много времени, но на его установку уйдет около минуты.

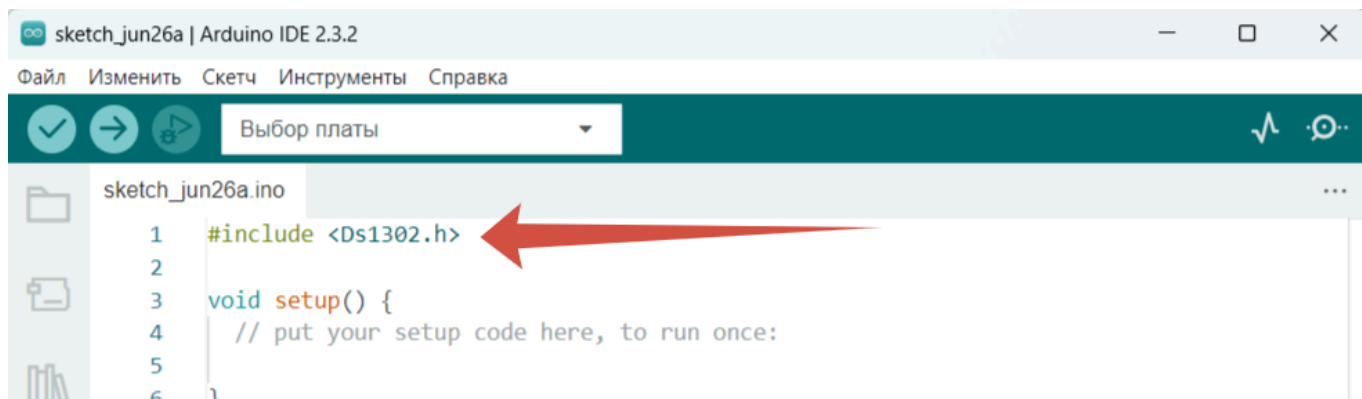


Когда все будет готово, мы сможем взглянуть на библиотеку в столбце менеджера библиотек, где должно быть написано «УСТАНОВЛЕНО» .

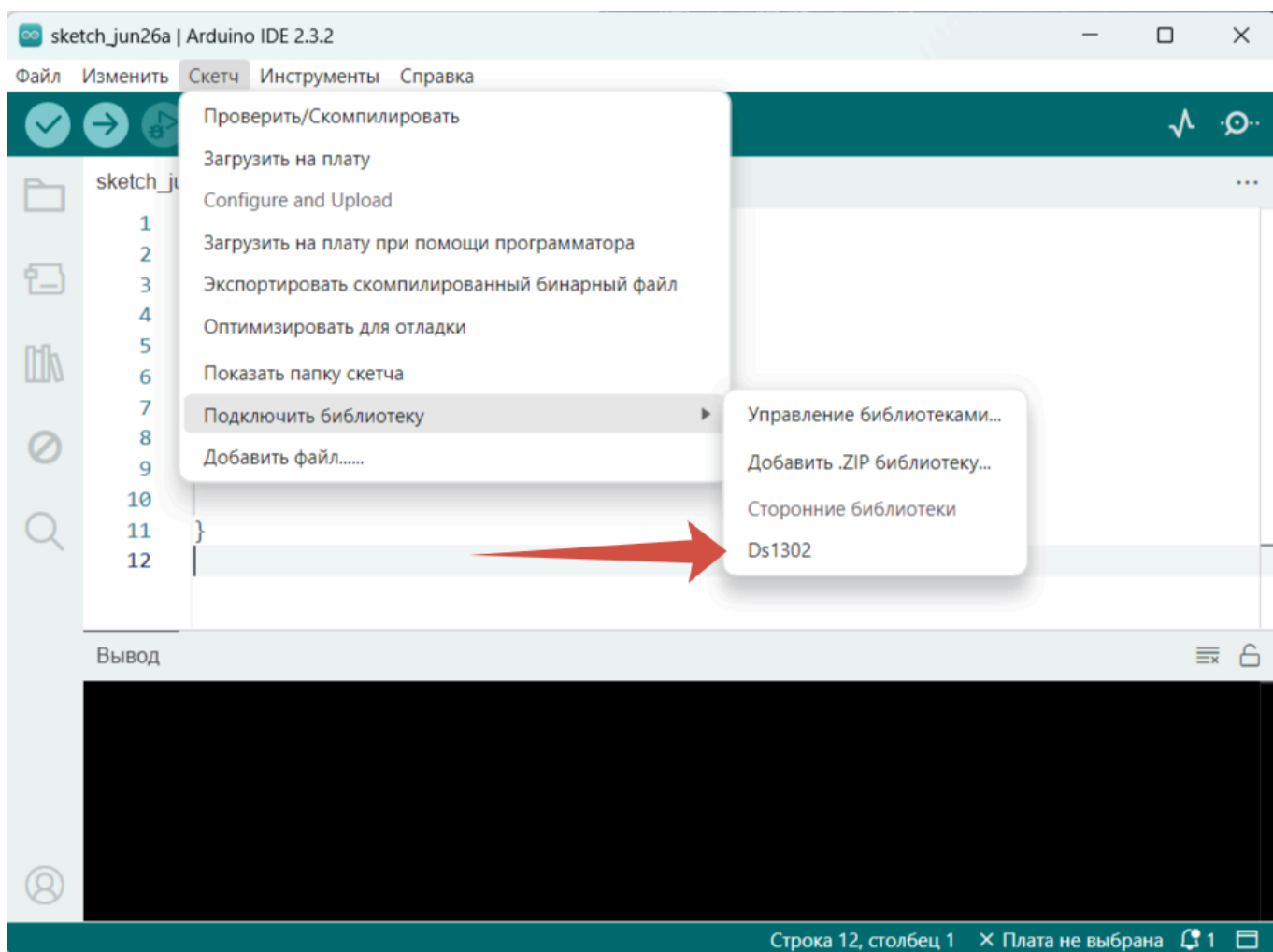


16.2 Подключение библиотеки

Чтобы использовать библиотеку, сначала необходимо включить ее в верхнюю часть эскиза.



Библиотеку можно напечатать или подключить выбрав из списка установленных библиотек.



более подробную информацию с можете ознакомиться на официальном сайте по ссылке <https://docs.arduino.cc/software/ide/#ide-v2>

2. Техника безопасности

При работе со схемами и электроникой важно соблюдать определенные правила безопасности, чтобы предотвратить травмы, повреждение оборудования или неисправности. Вот некоторые основные правила:

1. **Отключение питания.** Перед тем как вносить изменения в схему или проводить измерения, всегда отключайте питание. Это предотвратит возможные удары током и защитит компоненты от повреждений из-за случайных коротких замыканий.
2. **Использование подходящих инструментов.** Инструменты должны быть предназначены для работы с электроникой и находиться в исправном состоянии. Поврежденные инструменты могут привести к несчастным случаям или повредить компоненты.
3. **Защита себя:** Защитные очки защитят ваши глаза от случайных брызг при пайке или от мелких деталей, которые могут вылететь при обрезке выводов. Перчатки помогут избежать ожогов и порезов.
4. **Работа в хорошо вентилируемом помещении.** При пайке могут выделяться вредные пары, особенно если используется припой с канифолью. Хорошая вентиляция поможет избежать вдыхания этих паров.
5. **Изучение компонентов:** Понимание характеристик и ограничений компонентов поможет избежать их неправильного использования, которое может привести к их повреждению или выходу из строя.
6. **Соблюдение полярности и напряжения.** Неправильное подключение компонентов, чувствительных к полярности (например, диодов и электролитических конденсаторов), может привести к их повреждению. Также важно не превышать максимально допустимое напряжение компонентов.
7. **Проверка схемы перед включением.** Проверьте все соединения, чтобы убедиться, что нет коротких замыканий, и что все компоненты установлены правильно и в соответствии со схемой.
8. **Использование предохранителей и защиты от перенапряжения.** Предохранители и устройства защиты от перенапряжения помогут защитить вашу схему от случайных скачков напряжения и токов.
9. **Избегание работы во влажных условиях.** Влага может вызвать короткое замыкание и увеличить риск поражения электрическим током. Всегда работайте в сухой среде.
10. **Безопасное хранение компонентов и инструментов.** Храните острые и опасные предметы в недоступном для детей и домашних животных месте. Это поможет предотвратить случайные травмы и повреждения.
11. **Обучение и консультации.** Если вы не уверены в своих действиях, лучше проконсультироваться с более опытным коллегой или обратиться к надежным

обучающим материалам. Это поможет избежать ошибок и потенциальных проблем.

Соблюдение этих правил поможет обеспечить безопасную и продуктивную работу с электронными схемами.

3. Мигающий светодиод

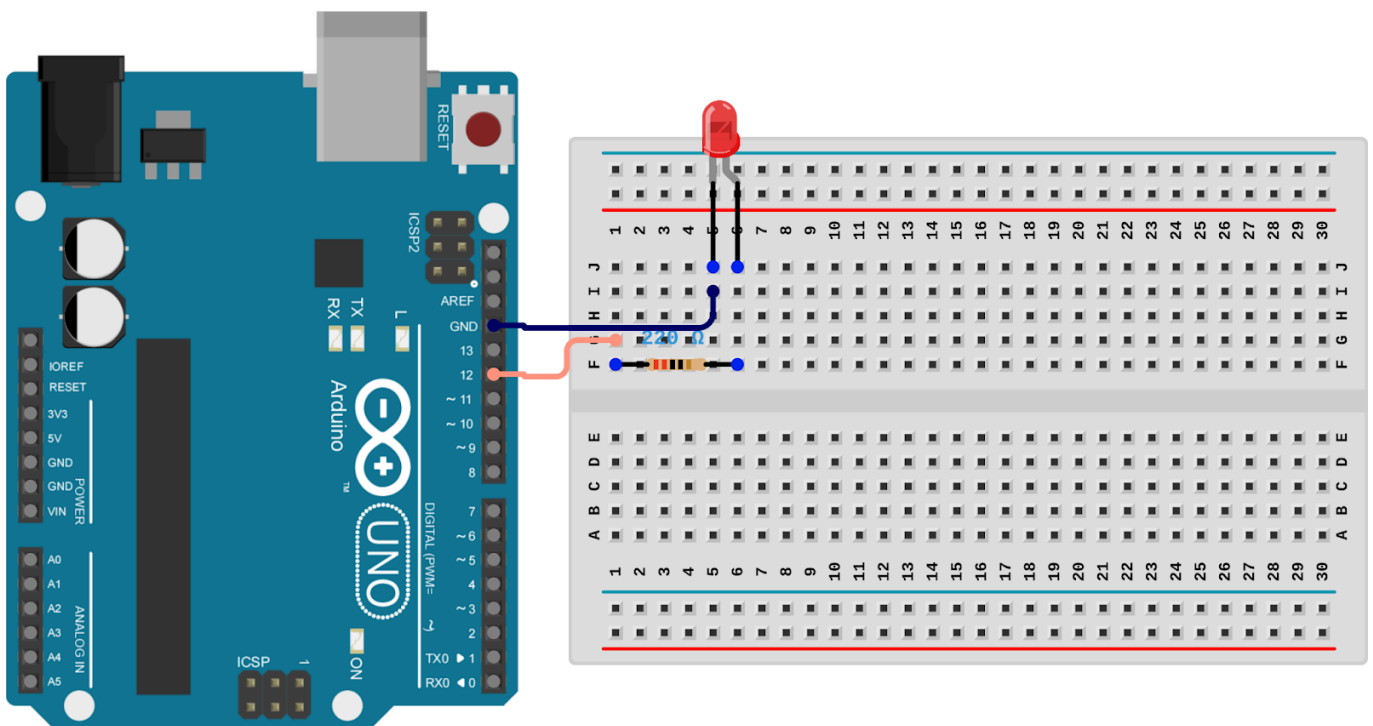
Создадим простую схему для демонстрации работы с цифровыми выходами Arduino UNO путём управления миганием светодиода.

Необходимые компоненты:

1. Arduino UNO
2. Красный светодиод
3. Резистор на 220 Ом
4. Провода для соединения компонентов

Инструкция по сборке схемы:

1. Вставьте резистор на 220 Ом в макетную плату.
2. Подключите анод (длинная ножка) светодиода к свободному концу резистора.
3. Подключите катод (короткая ножка) светодиода к одному из заземляющих (GND) выводов на Arduino UNO.
4. Подключите другой конец резистора к цифровому выводу D12 на Arduino UNO.



Код программы:

```
int led = 12; // Контакт 12 подключен к светодиоду через резистор
```

```

void setup() {
    pinMode(led, OUTPUT); // Инициализация цифрового вывода как
    ВЫХОД
}

void loop() {
    digitalWrite(led, HIGH); // Включить светодиод
    delay(1000);             // Подождать секунду
    digitalWrite(led, LOW);  // Выключить светодиод
    delay(1000);             // Подождать секунду
}

```

Принцип работы схемы:

Принцип работы данной схемы основан на управлении светодиодом с помощью микроконтроллера Arduino UNO. Схема использует цифровой выход микроконтроллера для переключения светодиода между включенным и выключенным состояниями, создавая эффект мигания.

Вот как это работает:

1. Инициализация. Когда Arduino UNO запускается или перезагружается, выполняется функция `setup()`. В этой функции цифровой вывод D12 настраивается как выход с помощью команды `pinMode(led, OUTPUT);`. Это означает, что микроконтроллер может управлять уровнем напряжения на этом выводе, устанавливая его либо в высокое (HIGH), либо в низкое (LOW) состояние.
2. Управление светодиодом. В основном цикле `loop()` микроконтроллер устанавливает вывод D12 в состояние HIGH с помощью команды `digitalWrite(led, HIGH);`. Когда вывод D12 находится в состоянии HIGH, через светодиод протекает ток, так как один его вывод (анод) подключен к D12 через резистор, а другой (катод) - к земле (GND). Это заставляет светодиод светиться.
3. Задержка. После включения светодиода программа ожидает 1000 миллисекунд (1 секунда), используя команду `delay(1000);`. В течение этого времени светодиод остается включенным.
4. Выключение светодиода. Затем микроконтроллер устанавливает вывод D12 в состояние LOW с помощью команды `digitalWrite(led, LOW);`. Когда вывод D12 находится в состоянии LOW, ток через светодиод не протекает, и он перестает светиться.

5. Повторение цикла. Программа снова ожидает 1000 миллисекунд (1 секунда) перед тем, как повторить цикл, снова включив светодиод. Этот процесс повторяется бесконечно, пока питание подается на Arduino UNO.

Роль резистора: Резистор на 220 Ом включен в цепь для ограничения тока, протекающего через светодиод, чтобы предотвратить его повреждение. Резистор создает падение напряжения и ограничивает ток до безопасного уровня для светодиода.

Таким образом, схема демонстрирует основы цифрового управления и принципы работы с электронными компонентами, такими как светодиоды и резисторы, а также позволяет практиковаться в программировании микроконтроллеров.

4. Диммируемый светодиод

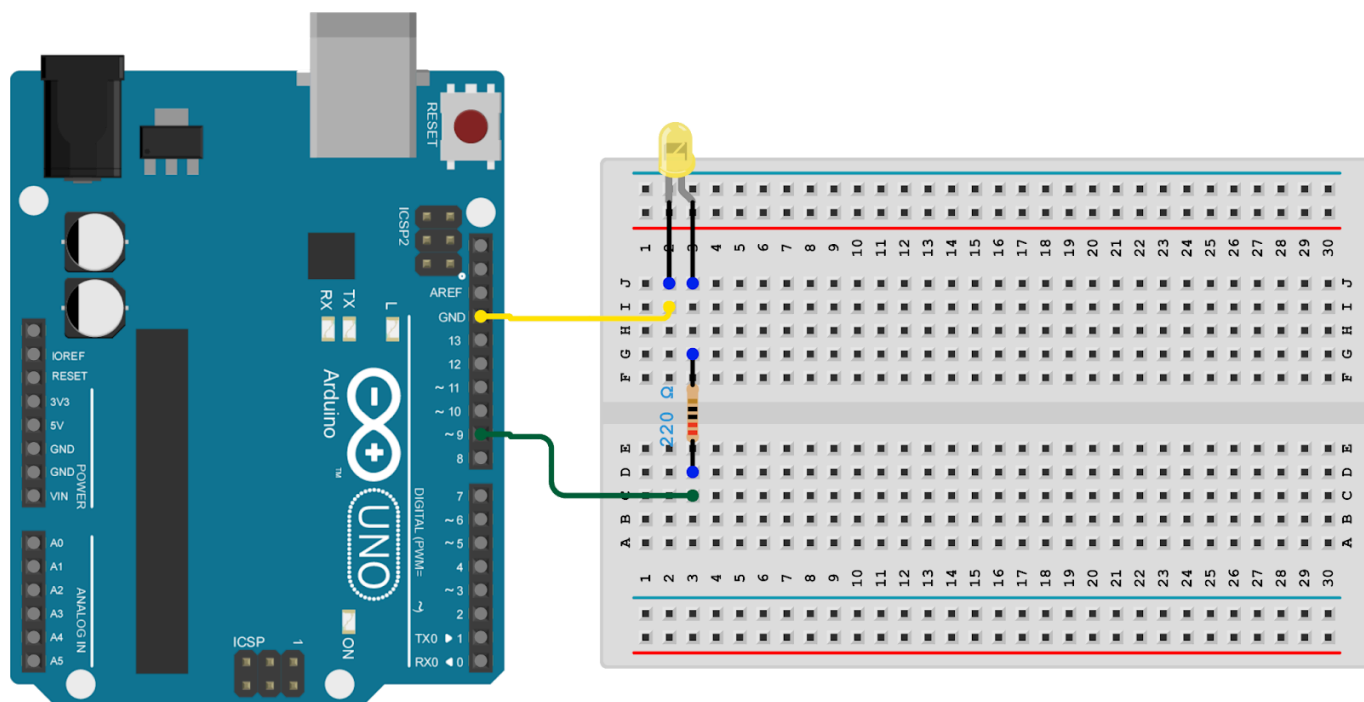
Создадим схему для демонстрации работы с аналоговыми выходами Arduino UNO, позволяющую управлять яркостью светодиода путём его плавного включения и выключения (диммирования).

Необходимые компоненты:

- Arduino UNO
- Жёлтый светодиод
- Резистор на 220 Ом
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Вставьте резистор на 220 Ом в макетную плату.
2. Подключите анод (длинная ножка) светодиода к свободному концу резистора.
3. Подключите катод (короткая ножка) светодиода к одному из заземляющих (GND) выводов на Arduino UNO.
4. Подключите другой конец резистора к цифровому выводу D9 на Arduino UNO, который настроен для работы с ШИМ-сигналом.



Код программы:

```
int led = 9;           // вывод светодиода 9
int brightness = 0;    // переменная, отвечающая за яркость
```

```

int fadeAmount = 5;    // переменная, которая задает скорость изменения
яркости за цикл

// настройки:
void setup() {
  pinMode(led, OUTPUT); // назначим вывод 9 выходом
}

// основной цикл:
void loop() {
  analogWrite(led, brightness); // устанавливаем яркость светодиода на
выводе 9

  brightness = brightness + fadeAmount; // изменяем значение яркости
для следующего прохода цикла

  if (brightness <= 0 || brightness >= 255) {
    fadeAmount = -fadeAmount; // меняем направление изменения яркости
  }
  delay(30); // ждем 30 миллисекунд для наблюдения эффекта
диммирования
}

```

Принцип работы схемы:

Схема использует ШИМ-выход (широтно-импульсная модуляция) микроконтроллера Arduino UNO для плавного изменения яркости светодиода. ШИМ позволяет изменять среднее напряжение, подаваемое на светодиод, путём быстрого переключения вывода между высоким и низким уровнями с различной скважностью.

Вот как это работает:

1. Инициализация. При запуске Arduino UNO функция `setup()` настраивает вывод D9 как выход с ШИМ-сигналом.
2. Диммирование. В функции `loop()` с помощью `analogWrite()` происходит управление яркостью светодиода, изменяя значение ШИМ-сигнала от 0 до 255 и обратно.
3. Задержка. Между изменениями яркости вставлена небольшая задержка (`delay(30)`), чтобы создать видимый эффект плавного перехода.

4. Роль резистора. Резистор на 220 Ом ограничивает ток через светодиод, предотвращая его повреждение.

Таким образом, схема демонстрирует использование ШИМ для создания эффекта диммирования светодиода и позволяет практиковаться в программировании микроконтроллеров с использованием аналоговых выходов.

5. Бегущая строка из светодиодов

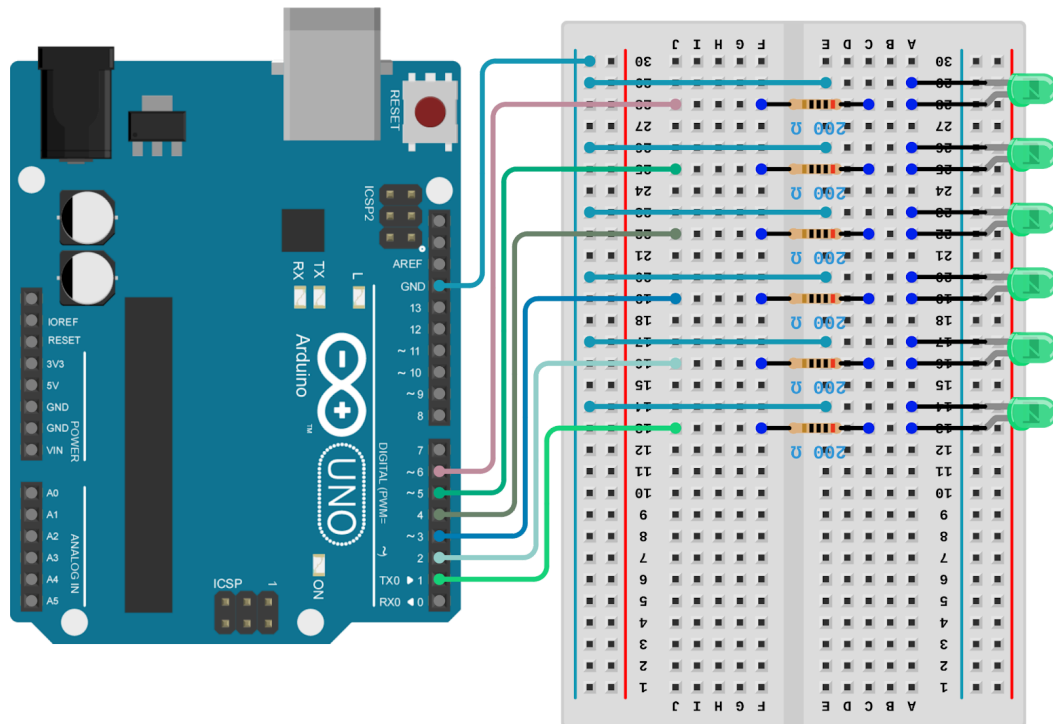
Создадим схему для демонстрации работы с цифровыми выходами Arduino UNO, управляя миганием шести светодиодов.

Необходимые компоненты:

- Arduino UNO
- 6 зелёных светодиодов
- 6 резисторов на 220 Ом
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Вставьте резисторы на 220 Ом в макетную плату.
2. Подключите аноды (длинные ножки) светодиодов к свободным концам резисторов.
3. Подключите катоды (короткие ножки) светодиодов к общему заземляющему (GND) выводу на Arduino UNO.
4. Подключите другие концы резисторов к цифровым выводам D1, D2, D3, D4, D5 и D6 на Arduino UNO.



Код программы:

```
// Определяем все 6 управляемых контактов на Arduino
```

```

int Led1 = 1;
int Led2 = 2;
int Led3 = 3;
int Led4 = 4;
int Led5 = 5;
int Led6 = 6;

void setup() {
    for(int i = 1; i <= 6; i++) // Пины 1-6 работают как выходные
        pinMode(i, OUTPUT); // i-й пин работает как выходной
}

void loop() {
    for(int j = 1; j <= 6; j++) { // Через цикл проходимся по всем 6 светодиодам
        digitalWrite(j, HIGH); // Каждому светодиоду присваиваем "HIGH", тем самым
он загорается
        delay(200); // Делаем задержку в 200 мс, чтобы включение было плавным
    }
    for(int j = 6; j >= 1; j--) { // Снова делаем цикл, чтобы пройти по
светодиодам и их выключить
        digitalWrite(j, LOW); // Каждому светодиоду в обратном порядке присваиваем
"LOW", тем самым его выключая
        delay(200); // Снова делаем задержку в 200 мс, чтобы выключение было
плавным
    }
}

```

Принцип работы схемы:

Схема управляет миганием шести светодиодов с помощью микроконтроллера Arduino UNO. Цифровые выходы микроконтроллера переключают светодиоды между включенным и выключенным состояниями, создавая эффект последовательного мигания.

Вот как это работает:

- Инициализация. При запуске Arduino UNO функция `setup()` настраивает пины D1-D6 как выходы.
- Управление светодиодами. В функции `loop()` микроконтроллер последовательно включает светодиоды, устанавливая высокий уровень напряжения (HIGH) на соответствующих пинах, и выключает их, устанавливая низкий уровень напряжения (LOW).
- Задержка. Между переключениями состояний светодиодов программа делает задержку в 200 миллисекунд для плавности мигания.

- Роль резисторов. Резисторы на 220 Ом ограничивают ток через светодиоды, предотвращая их повреждение.

Таким образом, схема демонстрирует использование нескольких выходов для управления светодиодами и позволяет практиковаться в программировании микроконтроллеров.

6. Индикатор на семисегментном дисплее

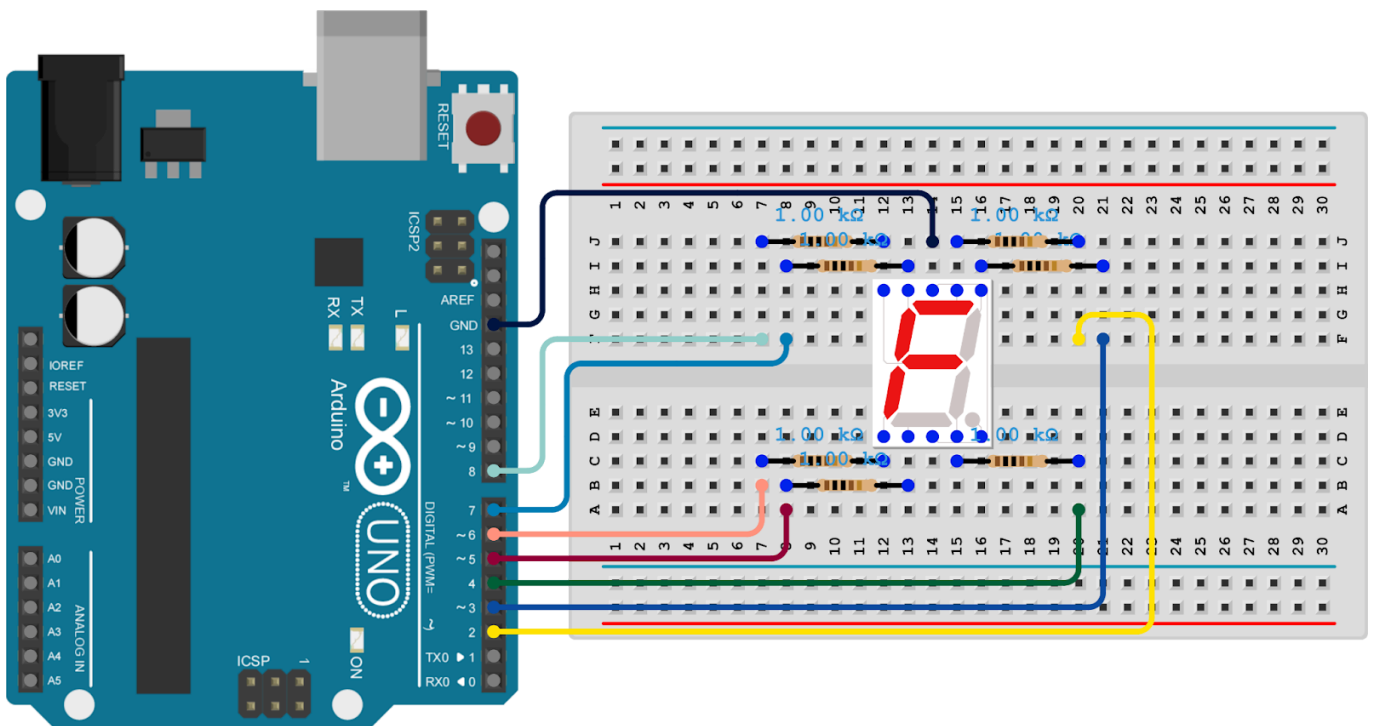
Создадим схему для демонстрации работы с цифровыми выходами Arduino UNO, управляя отображением на семисегментном индикаторе.

Необходимые компоненты:

- Arduino UNO
- Семисегментный индикатор с общим катодом
- Резисторы на 1 кОм (8 штук)
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите общий катод семисегментного индикатора к земле (GND) Arduino UNO.
2. Подключите каждый из сегментов индикатора (A-G) через резистор на 1 кОм к соответствующим цифровым выводам Arduino (D2-D8).
3. Подключите резисторы к цифровым выводам Arduino UNO следующим образом:
 - D2 к сегменту A
 - D3 к сегменту B
 - D4 к сегменту C
 - D5 к сегменту D
 - D6 к сегменту E
 - D7 к сегменту F
 - D8 к сегменту G



Код программы:

```
// Определение номеров выводов для сегментов семисегментного индикатора
const int led2 = 2; // Сегмент A
const int led3 = 3; // Сегмент B
const int led4 = 4; // Сегмент C
const int led5 = 5; // Сегмент D
const int led6 = 6; // Сегмент E
const int led7 = 7; // Сегмент F
const int led8 = 8; // Сегмент G

void setup() {
    // Настройка выводов как выходов для управления сегментами индикатора
    pinMode(led2, OUTPUT);
    pinMode(led3, OUTPUT);
    pinMode(led4, OUTPUT);
    pinMode(led5, OUTPUT);
    pinMode(led6, OUTPUT);
    pinMode(led7, OUTPUT);
    pinMode(led8, OUTPUT);
}

void loop() {
    int i = 200; // Время задержки в миллисекундах

    // Последовательное включение и выключение сегментов индикатора
    digitalWrite(led2, HIGH); // Включение сегмента A
    delay(i);                  // Задержка
```

```

digitalWrite(led2, LOW); // Выключение сегмента A
delay(i);                // Задержка

digitalWrite(led3, HIGH); // Включение сегмента B
delay(i);                // Задержка
digitalWrite(led3, LOW);  // Выключение сегмента B
delay(i);                // Задержка

digitalWrite(led4, HIGH); // Включение сегмента C
delay(i);                // Задержка
digitalWrite(led4, LOW);  // Выключение сегмента C
delay(i);                // Задержка

digitalWrite(led5, HIGH); // Включение сегмента D
delay(i);                // Задержка
digitalWrite(led5, LOW);  // Выключение сегмента D
delay(i);                // Задержка

digitalWrite(led6, HIGH); // Включение сегмента E
delay(i);                // Задержка
digitalWrite(led6, LOW);  // Выключение сегмента E
delay(i);                // Задержка

digitalWrite(led7, HIGH); // Включение сегмента F
delay(i);                // Задержка
digitalWrite(led7, LOW);  // Выключение сегмента F
delay(i);                // Задержка

digitalWrite(led8, HIGH); // Включение сегмента G
delay(i);                // Задержка
digitalWrite(led8, LOW);  // Выключение сегмента G
delay(i);                // Задержка
}

```

Принцип работы схемы:

Схема управляет отображением на семисегментном индикаторе, последовательно зажигая и гася каждый сегмент. Резисторы на 1000 Ом используются для ограничения тока через каждый сегмент, предотвращая их перегорание.

Вот как это работает:

- Инициализация. В функции `setup()` каждый цифровой вывод, к которому подключен сегмент индикатора, настраивается как выход.

- Управление сегментами. В функции `loop()` микроконтроллер последовательно устанавливает высокий уровень напряжения (HIGH) на выводах D2-D8, заставляя соответствующие сегменты светиться. Затем устанавливается низкий уровень напряжения (LOW), и сегменты гаснут. Каждое состояние длится 200 миллисекунд.
- Повторение цикла. Этот процесс повторяется бесконечно, создавая эффект "бегущего огня" по сегментам индикатора.

Таким образом, схема демонстрирует использование множественных выходов микроконтроллера для управления индикацией и позволяет практиковаться в программировании и работе с электронными компонентами.

В папке с кодом вы сможете найти еще одну программу, которая используя данную схему подключения, перебирает цифры от 0 до 9.

7. Управление светодиодами через регистр сдвига

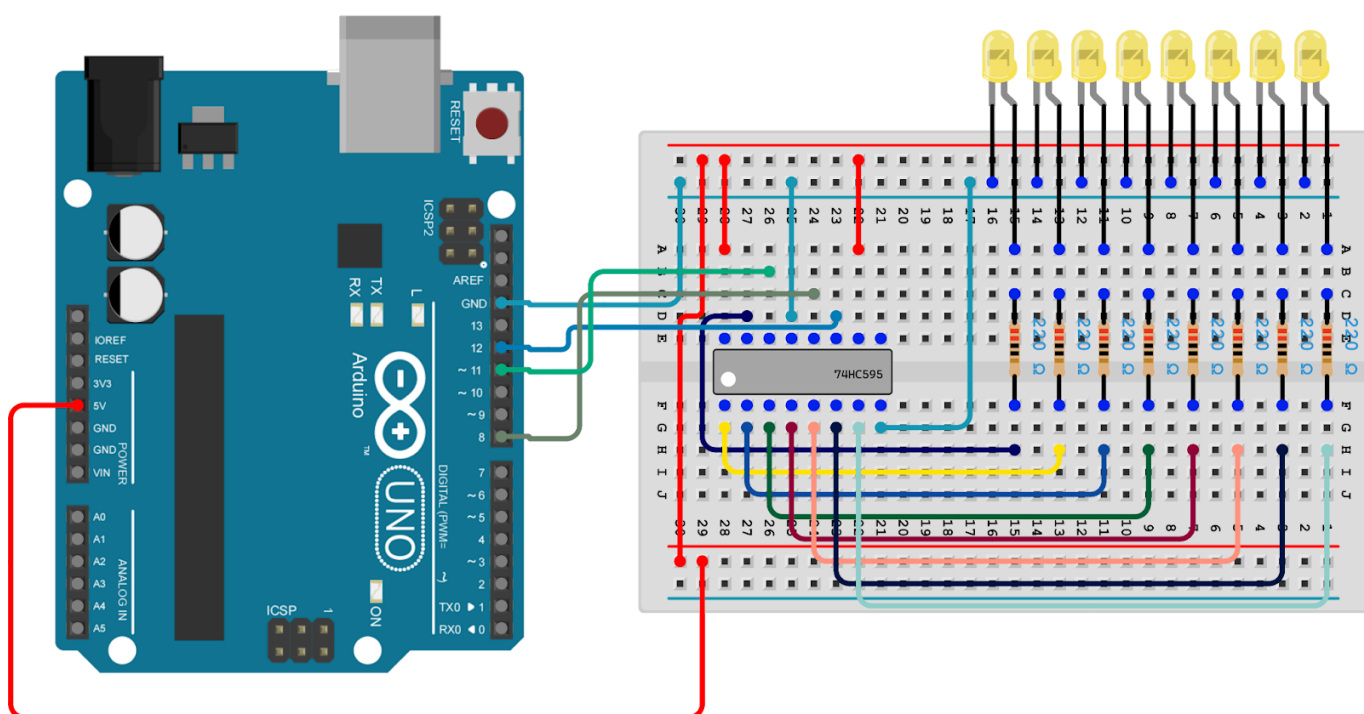
Создадим схему для демонстрации работы с цифровыми выходами Arduino UNO и регистром сдвига 74HC595, позволяющую управлять восемью светодиодами.

Необходимые компоненты:

- Arduino UNO
- Микросхема регистра сдвига 74HC595
- Восемь желтых светодиодов
- Восемь резисторов на 220 Ом
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите все катоды светодиодов к GND Arduino UNO.
2. Подключите аноды светодиодов через резисторы на 220 Ом к выходам Q0-Q7 микросхемы 74HC595.
3. Подключите VCC и MR (SRCLR) микросхемы 74HC595 к 5V Arduino UNO.
4. Подключите GND и OE (Enable) микросхемы 74HC595 к GND Arduino UNO.
5. Подключите пины SH_CP (SRCLK), ST_CP (RCLK), и DS (DATA) микросхемы 74HC595 к пинам D12, D8, и D11 Arduino UNO соответственно.



Код программы:

```
// Определение номеров пинов Arduino, подключенных к микросхеме 74HC595
int latchPin = 8; // Пин, подключенный к ST_CP (RCLK) регистра сдвига
int clockPin = 12; // Пин, подключенный к SH_CP (SRCLK) регистра сдвига
int dataPin = 11; // Пин, подключенный к DS (DATA) регистра сдвига

void setup() {
    // Настраиваем пины микроконтроллера как выходы
    pinMode(latchPin, OUTPUT);
    pinMode(clockPin, OUTPUT);
    pinMode(dataPin, OUTPUT);
    // Инициализируем последовательное соединение для отладки
    Serial.begin(9600);
}

void loop() {
    // Перебираем числа от 0 до 255 для отображения на светодиодах
    for (int numberToDisplay = 0; numberToDisplay < 256; numberToDisplay++) {
        // Выводим текущее число в последовательный порт
        Serial.print(numberToDisplay);
        Serial.print(" => ");
        // Выводим двоичное представление числа
        Serial.println(numberToDisplay, BIN);

        // Опускаем latchPin перед началом передачи данных
        digitalWrite(latchPin, LOW);
        // Отправляем биты в регистр сдвига
        shiftOut(dataPin, clockPin, LSBFIRST, numberToDisplay);
        // Поднимаем latchPin после передачи данных для активации выходов регистра
        digitalWrite(latchPin, HIGH);

        // Задержка между обновлениями светодиодов
        delay(1000);
    }
}
```

Принцип работы схемы:

Схема использует регистр сдвига 74HC595 для расширения количества цифровых выходов Arduino UNO, позволяя управлять восьмью светодиодами с использованием только трех пинов микроконтроллера. Каждый светодиод подключен к своему выходу регистра сдвига через резистор, который ограничивает ток и защищает светодиод от перегорания.

В коде программы используется функция `shiftOut()` для отправки последовательности битов в регистр сдвига, которые затем выводятся на светодиоды. Последовательность битов определяет, какие светодиоды будут включены или выключены. В данном примере кода светодиоды будут включаться по порядку от Q0 до Q7, создавая эффект "бегущего огонька".

Резисторы на 220 Ом используются для ограничения тока через светодиоды, предотвращая их повреждение. Подключение OE (Enable) к земле активирует выходы регистра сдвига, а подключение MR (SRCLR) к 5V предотвращает сброс регистра при включении питания.

8. Подключение Кнопки

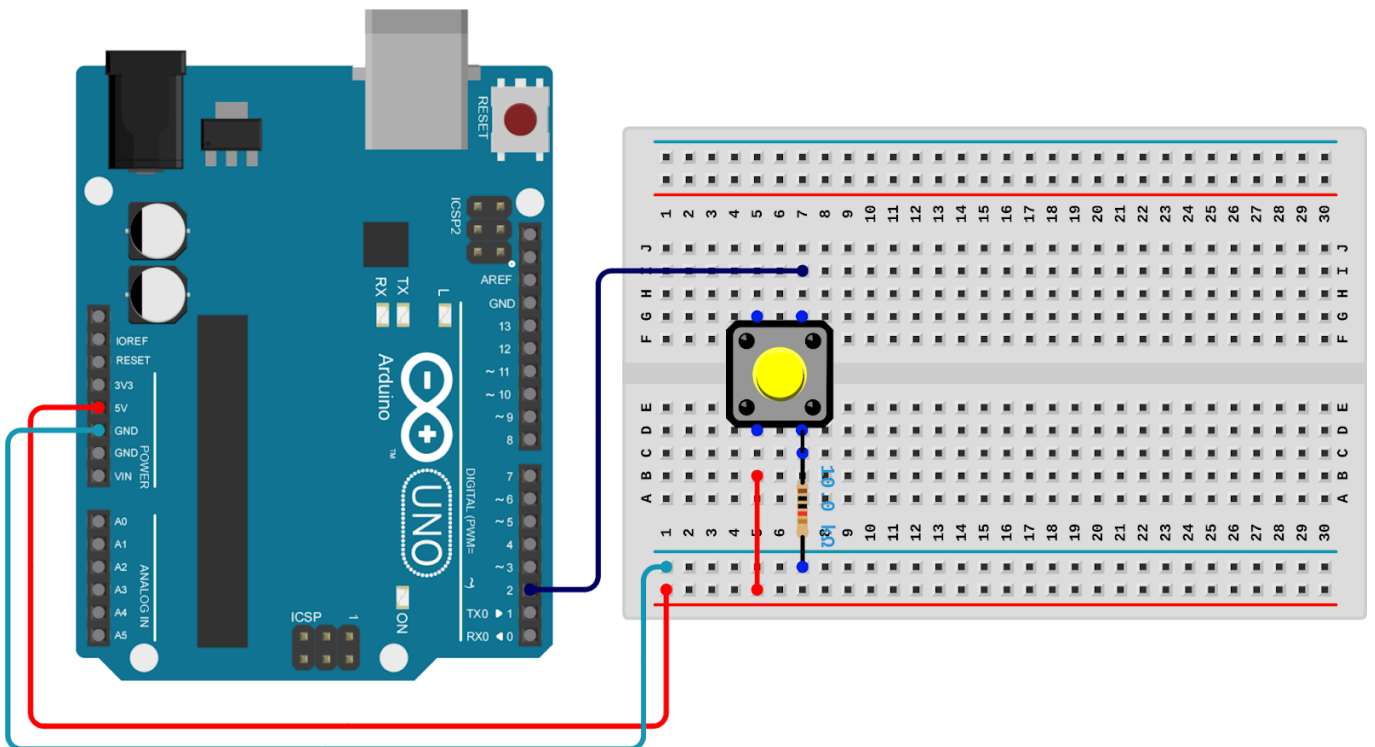
Рассмотрим, как подключить и использовать кнопку с микроконтроллером Arduino UNO для создания простого пользовательского интерфейса.

Необходимые компоненты:

- Arduino UNO
- Кнопка (Pushbutton)
- Резистор 10 кОм (подтягивающий)
- Провода для подключения

Инструкция по сборке схемы:

1. Подключите один контакт кнопки к цифровому входу D2 на Arduino.
2. Подключите другой контакт кнопки к 5V на Arduino.
3. Подключите один контакт резистора к земле (GND) на Arduino.
4. Подключите другой контакт резистора к тому же контакту кнопки, что и цифровой вход D2.



Код программы

```
// Цифровой вход 2 присоединен к кнопке. Объявим переменную для этого пина:
int pushButton = 2;

// Настройка, выполняемая один раз при запуске программы:
void setup() {
    // Инициализируем последовательную передачу данных со скоростью 9600 бит в секунду:
    Serial.begin(9600);
    // Назначим пин 2 входом:
    pinMode(pushButton, INPUT);
}

// Основной цикл программы, выполняется повторно:
void loop() {
    // Читаем значение на входном пине:
    int buttonState = digitalRead(pushButton);
    // Выводим значение на монитор последовательного порта:
    Serial.println(buttonState);
    // Задержка для стабильного считывания:
    delay(1);
}
```

Принцип работы схемы

Принцип работы данной схемы основан на использовании кнопки в качестве цифрового входа для микроконтроллера Arduino UNO. Схема реализует простой цифровой вход с использованием подтягивающего резистора. Состояние кнопки будем выводить на экран компьютера.

Вот как это работает:

1. Подтягивающий резистор. Резистор на 10 кОм подключен между землей (GND) и одним из контактов кнопки. Это обеспечивает устойчивое низкое состояние (0V, логический "0") на входе D2 Arduino, когда кнопка не нажата.
2. В начале работы (в функции setup) настраивается серийная передача данных со скоростью 9600 бит в секунду.
3. Состояние кнопки. Когда кнопка не нажата, через неё не течет ток, и вход D2 подтянут к земле через резистор, что обеспечивает низкий уровень сигнала.

Когда кнопка нажата, она замыкает цепь между 5V и входом D2, подтягивая его к высокому уровню напряжения (5V, логический "1").

4. Чтение состояния. Микроконтроллер в цикле loop считывает состояние входа D2 с помощью функции `digitalRead(pushButton)`. Если кнопка нажата, функция вернет LOW (0), если отпущена - HIGH (1).
5. Вывод данных. Состояние в серийный порт с помощью `Serial.println(buttonState)`, позволяя наблюдать за изменениями состояния кнопки в реальном времени через монитор последовательного порта в Arduino IDE.

Таким образом, схема позволяет детектировать нажатие кнопки и передавать это состояние в компьютер для отображения или дальнейшей обработки.

9. Управление светодиодом и кнопкой

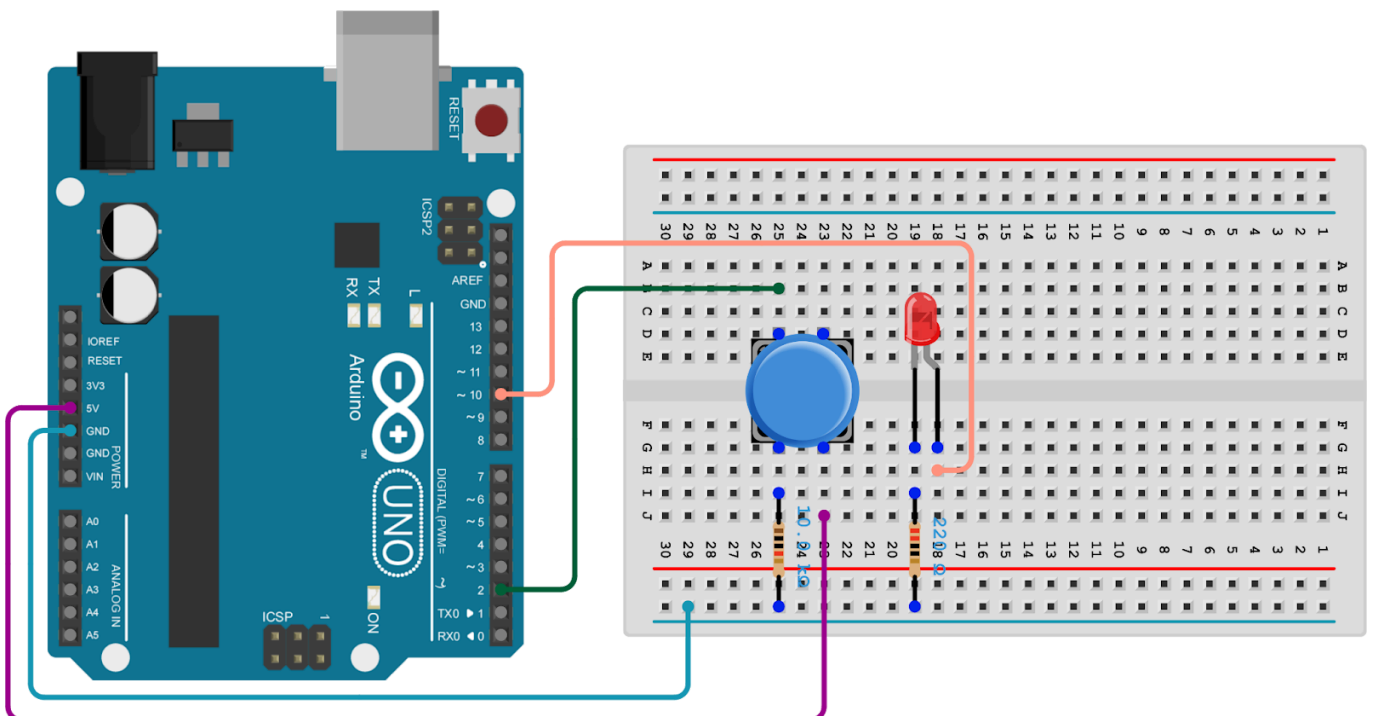
Создадим схему, позволяющую управлять включением светодиода с помощью кнопки, используя цифровые входы и выходы Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Красный светодиод
- Резистор на 220 Ом
- Резистор на 10 кОм
- Кнопка
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите резистор на 220 Ом к аноду (длинная ножка) светодиода.
2. Подключите катод (короткая ножка) светодиода к земле (GND) Arduino UNO.
3. Подключите другой конец резистора на 220 Ом к цифровому выводу D10 на Arduino UNO.
4. Подключите один из выводов кнопки к цифровому выводу D2 на Arduino UNO.
5. Подключите другой вывод кнопки к питанию 5V на Arduino UNO.
6. Подключите резистор на 10 кОм между выводом D2 и землей (GND), образуя тем самым подтягивающий резистор.



Код программы:

```
const int LED = 10; // вывод для подключения светодиода (D10)
const int Button = 2; // вход для подключения кнопки (D2)

void setup() {
    pinMode(LED, OUTPUT); // Конфигурируем вывод подключения светодиода как выход
    pinMode(Button, INPUT); // Конфигурируем вывод подключения кнопки как вход
}

void loop() {
    int buttonState = digitalRead(Button); // Считываем состояние кнопки
    if (buttonState) {
        digitalWrite(LED, HIGH); // Если кнопка нажата, включаем светодиод
    } else {
        digitalWrite(LED, LOW); // Если кнопка отжата, выключаем светодиод
    }
}
```

Принцип работы схемы:

Схема позволяет управлять светодиодом при помощи кнопки. Когда кнопка не нажата, подтягивающий резистор на 10 кОм удерживает вход D2 в состоянии LOW, и светодиод выключен. При нажатии кнопки вход D2 подключается к 5V, считывается состояние HIGH, и светодиод загорается. Это демонстрирует использование цифровых входов/выходов и простую обработку входных сигналов с помощью микроконтроллера.

10. Чтение аналогового сигнала с потенциометра

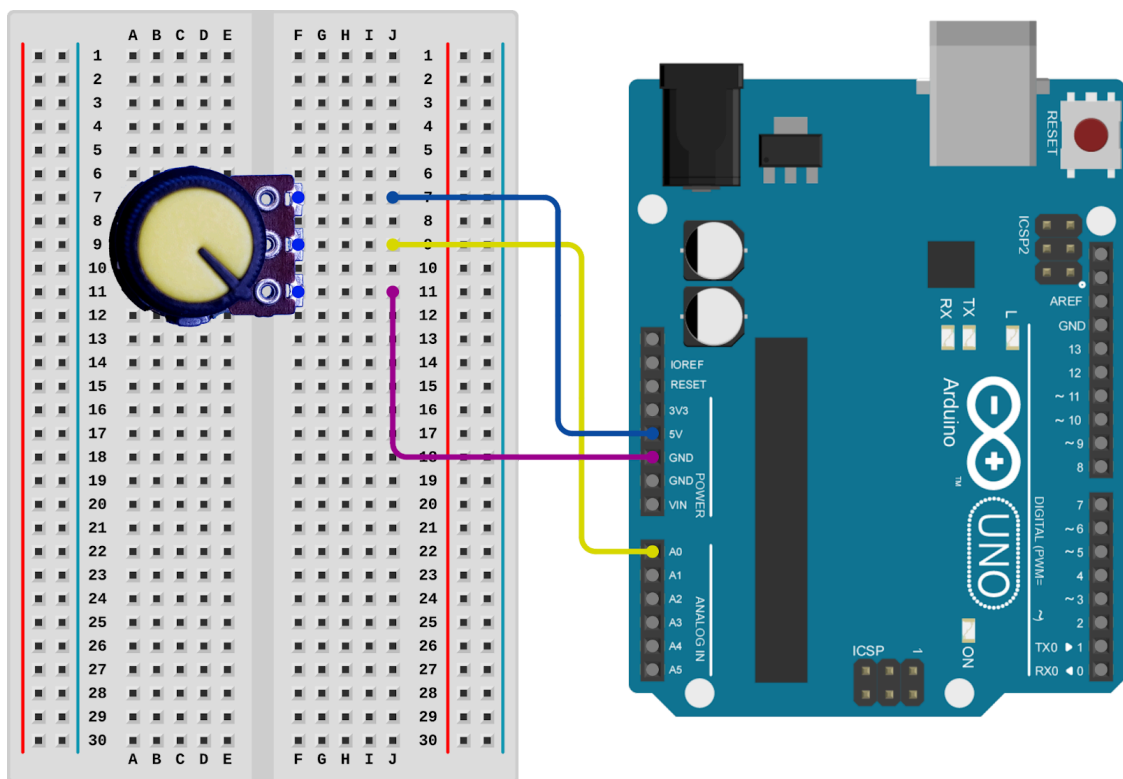
Создадим схему для демонстрации работы с аналоговыми входами Arduino UNO, используя потенциометр для ввода переменного напряжения.

Необходимые компоненты:

- Arduino UNO
- Потенциометр
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите один из внешних контактов потенциометра к земле (GND) на Arduino UNO.
2. Подключите другой внешний контакт потенциометра к 5V на Arduino UNO.
3. Подключите средний контакт потенциометра (вывод сигнала) к аналоговому входу A0 на Arduino UNO.



Код программы:

```
void setup() {  
    Serial.begin(9600); // Инициализация последовательного порта со скоростью 9600  
    бод
```

```

}

void loop() {
    int sensorValue = analogRead(A0); // Чтение аналогового значения с
    потенциометра
    Serial.println(sensorValue);      // Вывод значения в монитор последовательного
    порта
    delay(1);                         // Задержка для стабилизации чтения
}

```

Принцип работы схемы:

Принцип работы данной схемы основан на преобразовании аналогового сигнала, который генерируется потенциометром, в цифровое значение с помощью аналогово-цифрового преобразователя (АЦП) в Arduino UNO.

Вот как это работает:

1. Инициализация. При запуске Arduino выполняется функция `setup()`, где инициализируется последовательный порт для общения с компьютером.
2. Чтение аналогового сигнала. В функции `loop()` с помощью функции `analogRead(A0)` происходит чтение аналогового значения с вывода A0, к которому подключен потенциометр.
3. Вывод данных. Считанное значение `sensorValue` отправляется в монитор последовательного порта с помощью `Serial.println(sensorValue)`, где его можно увидеть.
4. Задержка. Функция `delay(1)` обеспечивает небольшую задержку между чтениями для стабильности показаний.

Поворачивая ручку потенциометра, вы изменяете сопротивление, что приводит к изменению напряжения на среднем контакте. Arduino UNO считывает это напряжение и преобразует его в цифровое значение от 0 до 1023, которое отображается в мониторе последовательного порта. Это позволяет наблюдать за изменением аналогового сигнала в реальном времени.

11. Двухосевой джойстик

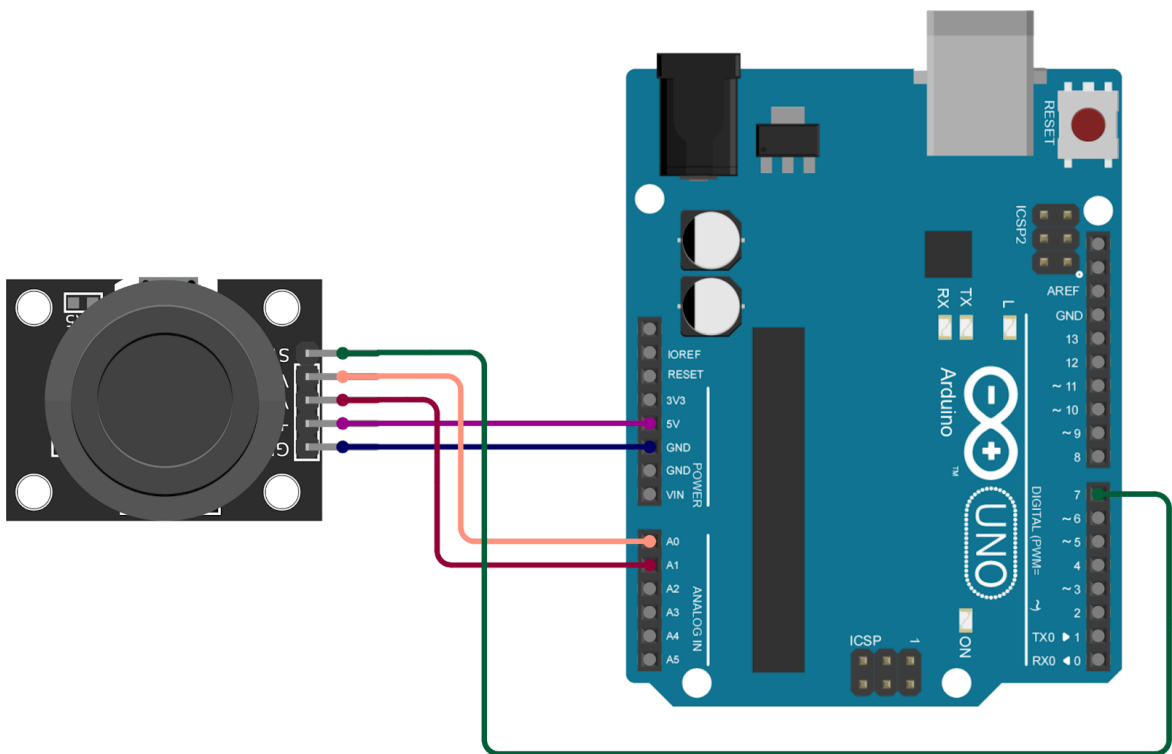
Создадим схему для чтения положения двухосевого джойстика и состояния его кнопки с помощью микроконтроллера Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Двухосевой джойстик KY-023
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите GND джойстика KY-023 к GND на Arduino UNO.
2. Подключите +5V джойстика KY-023 к 5V на Arduino UNO.
3. Подключите вывод VRx джойстика к аналоговому входу A1 на Arduino UNO.
4. Подключите вывод VRy джойстика к аналоговому входу A0 на Arduino UNO.
5. Подключите вывод SW джойстика к цифровому входу D7 на Arduino UNO.



Код программы:

```
const int axisX = A0; // ось X подключена к A0
const int axisY = A1; // ось Y подключена к A1
const int buttonPin = 7; // кнопка джойстика подключена к D7
int valX, valY, valButton = 0; // переменные для хранения значений осей и кнопки
```



```

void setup() {
    pinMode(buttonPin, INPUT_PULLUP); // конфигурируем D7 как INPUT с включением
    подтягивающего резистора
    Serial.begin(9600);
}

void loop() {
    valX = analogRead(axisX); // значение оси X
    valY = analogRead(axisY); // значение оси Y
    valButton = 1 - digitalRead(buttonPin); // значение кнопки (инвертированное)
    // выводим значения в монитор порта
    Serial.print("X:"); Serial.print(valX, DEC);
    Serial.print(" | Y:"); Serial.print(valY, DEC);
    Serial.print(" | Button: "); Serial.println(valButton, DEC);
    delay(500); // пауза перед следующим считыванием данных
}

```

Принцип работы схемы:

Схема позволяет считывать аналоговые значения положения джойстика по осям X и Y, а также состояние кнопки. Аналоговые значения осей X и Y меняются в зависимости от положения ручки джойстика, а состояние кнопки изменяется при её нажатии.

Вот как это работает:

- Инициализация. В функции `setup()` настраивается цифровой вход для кнопки с подтягивающим резистором и инициализируется последовательное соединение.
- Чтение значений. В функции `loop()` происходит чтение аналоговых значений осей джойстика с помощью функций `analogRead(axisX)` и `analogRead(axisY)`. Состояние кнопки считывается как цифровое значение с помощью `digitalRead(buttonPin)` и инвертируется, так как кнопка подключена таким образом, что при нажатии она замыкается на землю.
- Вывод данных. Считанные значения выводятся в монитор порта для наблюдения.
- Задержка. Между итерациями цикла `loop()` устанавливается задержка в 500 миллисекунд для удобства чтения данных.

Таким образом, данная схема позволяет ознакомиться с аналоговым и цифровым вводом данных в Arduino, а также с основами работы с джойстиком и обработкой его сигналов.

12. Генератор звуковой частоты

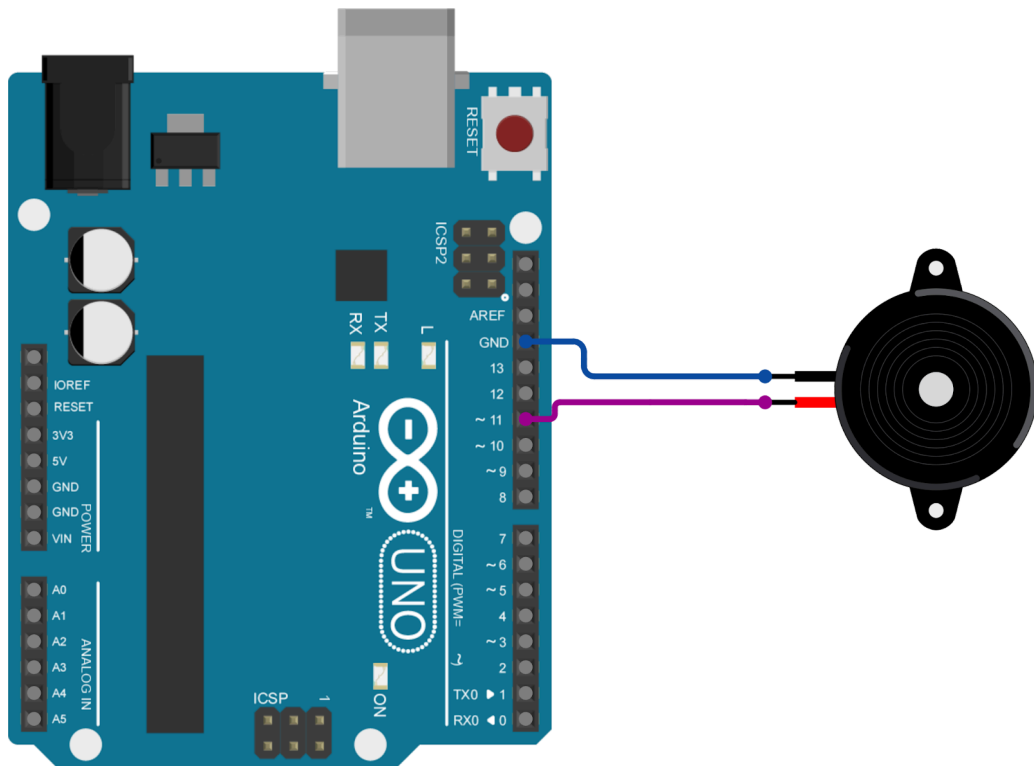
Создадим схему, которая позволяет генерировать звуковые частоты с помощью пьезоизлучателя, управляемого микроконтроллером Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Пьезоизлучатель (пьезодинамик)

Инструкция по сборке схемы:

1. Подключите один контакт пьезоизлучателя к цифровому выводу D11 на Arduino UNO.
2. Подключите другой контакт пьезоизлучателя к заземляющему (GND) выводу на Arduino UNO.



Код программы:

```
int buzzer = 11; // Пин 11 подключен к пьезоизлучателю

void setup() {
  pinMode(buzzer, OUTPUT); // Инициализация цифрового вывода как выход
}
```

```

void loop() {
  unsigned char i, j; // Задаем переменные
  while(1) { // Бесконечный цикл
    for(i = 1; i < 80; i++) { // Первая частота звука
      digitalWrite(buzzer, HIGH); // Пьезоизлучатель издает звук
      delay(1); // Задержка 1 мс
      digitalWrite(buzzer, LOW); // Пьезоизлучатель не издает звук
      delay(1); // Задержка 1 мс
    }
    for(i = 0; i < 100; i++) { // Вторая частота звука
      digitalWrite(buzzer, HIGH); // Пьезоизлучатель издает звук
      delay(2); // Задержка 2 мс
      digitalWrite(buzzer, LOW); // Пьезоизлучатель не издает звук
      delay(2); // Задержка 2 мс
    }
  }
}

```

Принцип работы схемы:

Схема использует цифровой выход микроконтроллера Arduino UNO для генерации звуковых колебаний, которые воспроизводятся пьезоизлучателем. Пьезоизлучатель подключен к цифровому выводу D11, который управляется программой, загруженной в микроконтроллер.

Вот как это работает:

- Инициализация. В функции setup() вывод D11 настраивается как выход.
- Генерация звука. В функции loop() микроконтроллер создает два различных звуковых сигнала путем быстрого переключения вывода D11 между состояниями HIGH и LOW. Первый сигнал генерируется с периодом 2 мс (1 мс HIGH и 1 мс LOW), а второй с периодом 4 мс (2 мс HIGH и 2 мс LOW). Это создает два различных тона.
- Бесконечный цикл. Цикл while(1) в функции loop() заставляет Arduino бесконечно повторять последовательность генерации звука.

Таким образом, схема демонстрирует использование Arduino для создания простого генератора звуковых частот, который может быть использован в различных образовательных и хоббийных проектах.

В папке с годом данного урока сможете найти дополнительный код, который генерирует одну из узнаваемых мелодий.

13. Игра "Быстрая дуэль"

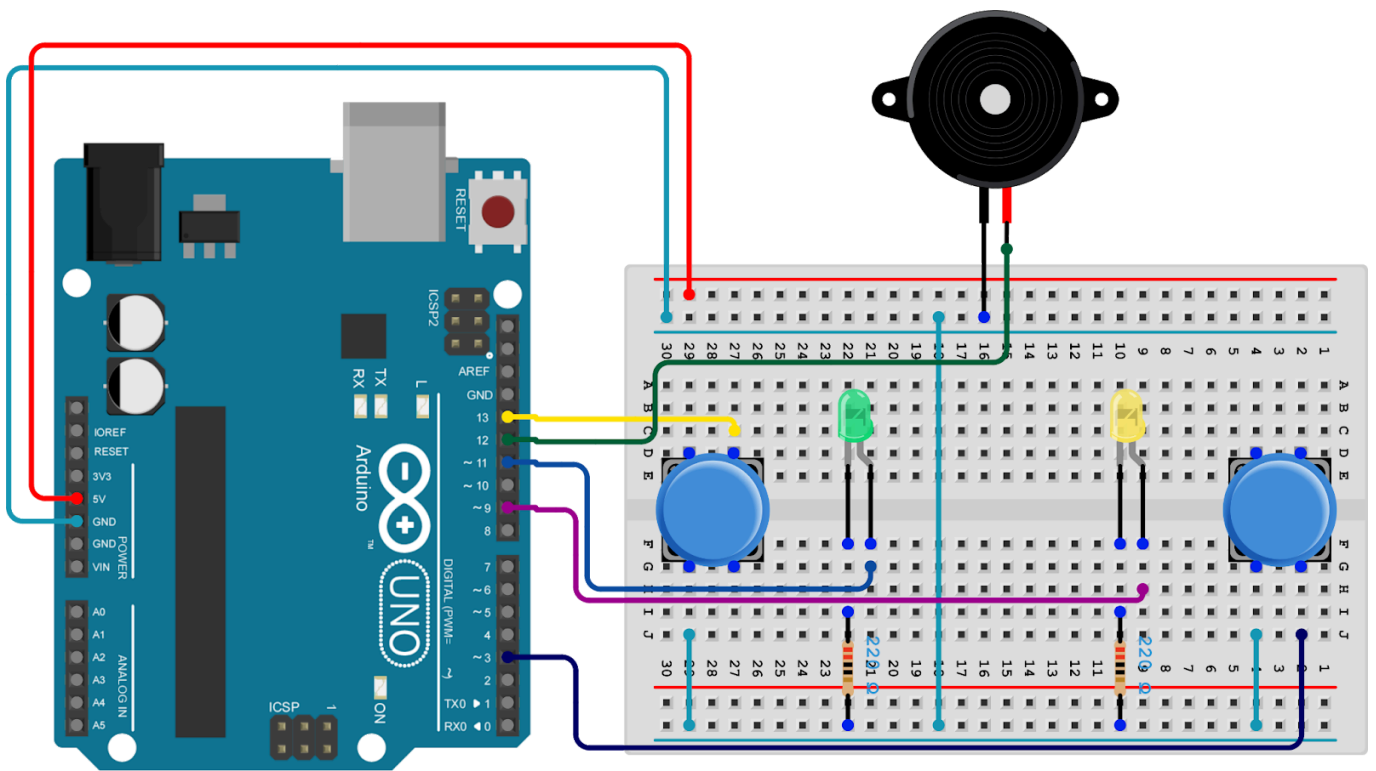
Создадим интерактивную игру на базе Arduino UNO, которая тестирует реакцию игроков. Игроки должны нажать на кнопку как можно быстрее после звукового сигнала.

Необходимые компоненты:

- Arduino UNO
- 2 зелёных светодиода
- 2 жёлтых светодиода
- 4 резистора на 220 Ом
- 2 кнопки
- Пьезоизлучатель
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите аноды зелёных и жёлтых светодиодов к цифровым выводам D9 и D11 Arduino UNO соответственно.
2. Подключите катоды светодиодов к земле (GND) через резисторы на 220 Ом.
3. Подключите один контакт каждой кнопки к цифровым выводам D3 и D13 Arduino UNO.
4. Подключите другой контакт кнопок к земле (GND) через резисторы на 220 Ом.
5. Подключите пьезоизлучатель к цифровому выводу D12 и к земле (GND).



Код программы:

```
#define BUZZER_PIN 12 // пин с пищалкой
#define PLAYER_COUNT 2 // количество игроков-ковбоев
// вместо перечисления всех пинов по-одному, мы объявляем пару
// списков: один с номерами пинов с кнопками, другой -- со
// светодиодами. Списки также называют массивами (англ. array)
int buttonPins[PLAYER_COUNT] = {3, 13};
int ledPins[PLAYER_COUNT] = {9, 11};

void setup()
{
    pinMode(BUZZER_PIN, OUTPUT);
    for (int player = 0; player < PLAYER_COUNT; ++player) {
        // при помощи квадратных скобок получают значение в массиве
        // под указанным в них номером. Нумерация начинается с нуля
        pinMode(ledPins[player], OUTPUT);
        pinMode(buttonPins[player], INPUT_PULLUP);
    }
}

void loop()
{
    // даём сигнал «пли!», выждав случайное время от 2 до 7 сек
    delay(random(2000, 7000));
    tone(BUZZER_PIN, 3000, 250); // 3 килогерца, 250 миллисекунд
```

```

for (int player = 0; ; player = (player+1) % PLAYER_COUNT) {
    // если игрок номер «player» нажал кнопку...
    if (!digitalRead(buttonPins[player])) {
        // ...включаем его светодиод и сигнал победы на 1 сек
        digitalWrite(ledPins[player], HIGH);
        tone(BUZZER_PIN, 4000, 1000);
        delay(1000);
        digitalWrite(ledPins[player], LOW);
        break; // Есть победитель! Выходим (англ. break) из цикла
    }
}
}
}

```

Принцип работы схемы:

Игра начинается с того, что Arduino UNO в случайный момент времени издает короткий звуковой сигнал через пьезоизлучатель. Игроки должны как можно быстрее нажать на свою кнопку после сигнала. Кнопки подключены с внутренней подтяжкой к питанию (INPUT_PULLUP), что означает, что их состояние по умолчанию — высокий уровень (HIGH), и они переходят в низкий уровень (LOW), когда кнопка нажата.

Когда один из игроков нажимает кнопку, соответствующий светодиод загорается, и пьезоизлучатель издает длинный звуковой сигнал, сигнализируя о победе. Затем светодиод выключается, и игра начинается заново.

Резисторы на 220 Ом используются для ограничения тока через светодиоды, предотвращая их перегорание. Пьезоизлучатель используется для генерации звуковых сигналов, информирующих игроков о начале раунда и о победе.

Таким образом, проект демонстрирует использование цифровых входов и выходов Arduino UNO, работу с кнопками, светодиодами и пьезоизлучателем, а также программирование случайных задержек и обработку входных сигналов.

14. Датчик освещенности на основе фоторезистора

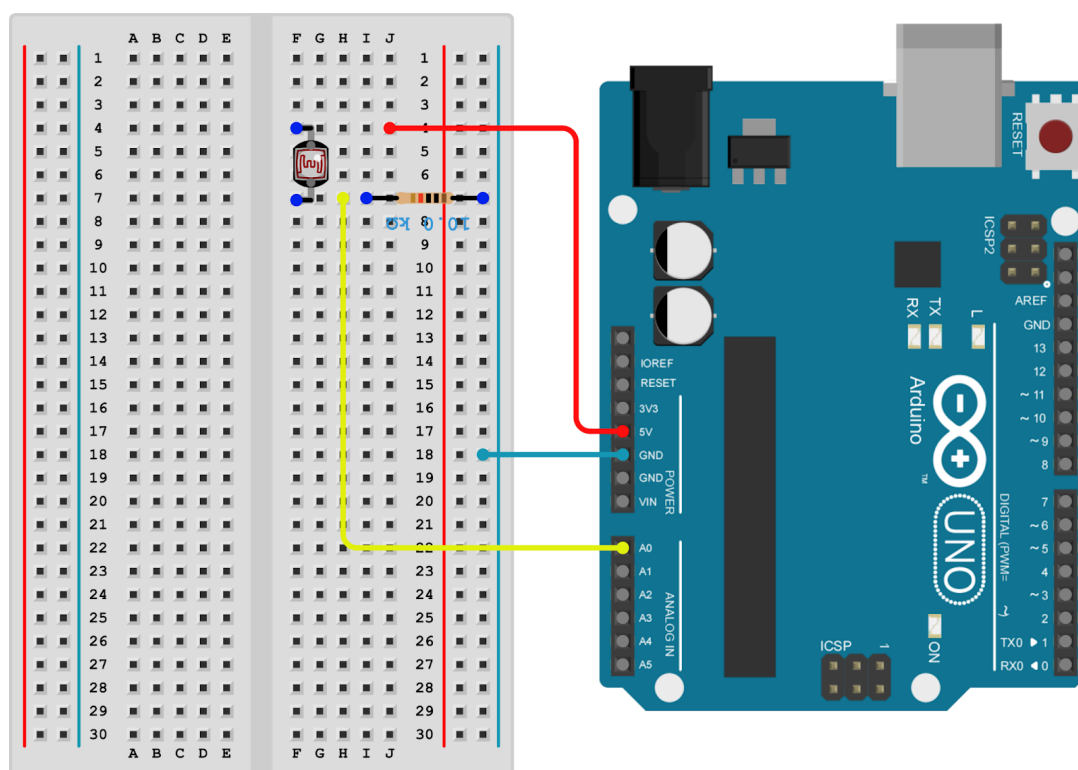
Создадим схему для измерения уровня освещенности с помощью фоторезистора и микроконтроллера Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Фоторезистор (LDR)
- Резистор на 10 кОм
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите один вывод фоторезистора к аналоговому входу A0 на Arduino UNO.
2. Подключите другой вывод фоторезистора к питанию 5V на Arduino UNO.
3. Подключите один вывод резистора на 10 кОм к аналоговому входу A0, где уже подключен фоторезистор.
4. Подключите другой вывод резистора к земле (GND) на Arduino UNO.



Код программы:

```
int light; // переменная для хранения данных фоторезистора
```

```
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  light = analogRead(0); // Чтение аналогового значения с фоторезистора
  Serial.println(light); // Вывод значения освещенности в монитор порта
  delay(100); // Задержка для стабилизации чтения
}
```

Принцип работы схемы:

Схема использует фоторезистор для измерения уровня освещенности. Фоторезистор изменяет свое сопротивление в зависимости от интенсивности падающего на него света. Вместе с резистором на 10 кОм он образует делитель напряжения, напряжение с которого подается на аналоговый вход A0 Arduino UNO.

Когда освещенность высокая, сопротивление фоторезистора уменьшается, и напряжение на A0 возрастает. Когда освещенность низкая, сопротивление фоторезистора увеличивается, и напряжение на A0 падает. Arduino UNO считывает это напряжение и преобразует его в цифровое значение, которое затем может быть использовано для оценки уровня освещенности.

В коде программы используется функция `analogRead(0)`, которая считывает аналоговое значение с пина A0, и значение выводится в монитор порта с помощью `Serial.println(light)`. Это позволяет наблюдать изменения освещенности в реальном времени.

15. Индикатор уровня освещенности

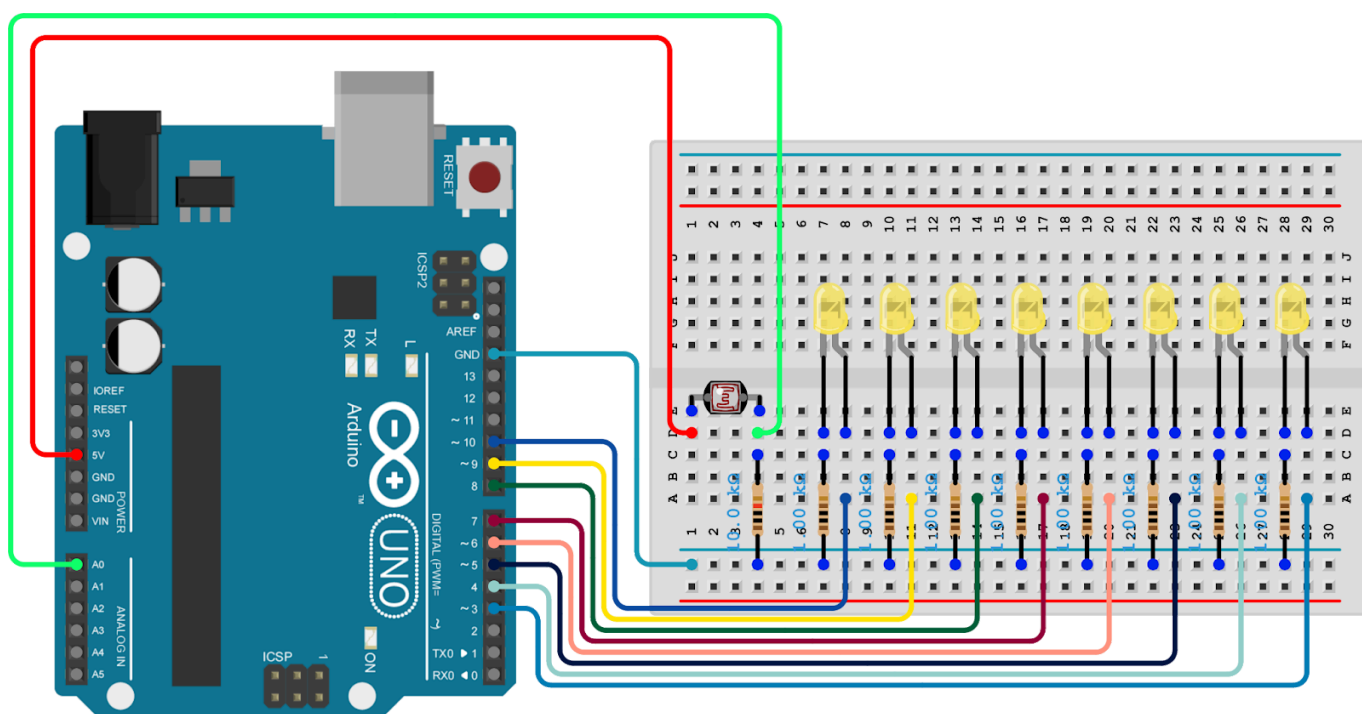
Создадим схему, позволяющую демонстрировать работу с аналоговыми и цифровыми выходами Arduino UNO для индикации уровня освещенности с помощью светодиодов.

Необходимые компоненты:

- Arduino UNO
- Фоторезистор (LDR)
- 10 светодиодов (желтые)
- 10 резисторов на 1 кОм
- 1 резистор на 10 кОм
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите фоторезистор к аналоговому входу A0 на Arduino UNO.
2. Подключите один конец фоторезистора к 5V на Arduino UNO.
3. Подключите другой конец фоторезистора к резистору на 10 кОм, а другой конец резистора к GND на Arduino UNO, создавая делитель напряжения.
4. Подключите аноды всех светодиодов к цифровым выводам D3-D10 на Arduino UNO через резисторы на 10 кОм.
5. Подключите катоды всех светодиодов к GND на Arduino UNO.



Код программы:

```
// Контакт подключения светодиодов
const int leds[] = {3, 4, 5, 6, 7, 8, 9, 10};
const int LIGHT = A0; // Контакт A0 для входа фоторезистора
const int MIN_LIGHT = 200; // Нижний порог освещенности
const int MAX_LIGHT = 900; // Верхний порог освещенности
// Переменная для хранения данных фоторезистора
int val = 0;

void setup() {
    // Сконфигурировать контакты светодиодов как выход
    for (int i = 0; i < 8; i++) // делаем через цикл for
        pinMode(leds[i], OUTPUT);
}

void loop() {
    val = analogRead(LIGHT); // Считываем показания фоторезистора
    // Применение функции map()
    val = map(val, MIN_LIGHT, MAX_LIGHT, 8, 0);
    // Ограничиваем, чтобы не превысило границ
    val = constrain(val, 0, 8);
    // Зажечь количество светодиодов, пропорциональное освещенности,
    // остальные потушить
    for (int i = 1; i < 9; i++) {
        if (i >= val) // Зажечь светодиоды
            digitalWrite(leds[i - 1], HIGH);
        else // Потушить светодиоды
            digitalWrite(leds[i - 1], LOW);
    }
    delay(1000); // Пауза перед следующим измерением
}
```

Принцип работы схемы:

Схема использует фоторезистор для измерения уровня освещенности. Фоторезистор изменяет свое сопротивление в зависимости от интенсивности света, падающего на него. Это изменение сопротивления приводит к изменению напряжения на аналоговом входе A0 Arduino UNO. Микроконтроллер считывает это напряжение и преобразует его в цифровое значение, которое затем используется для управления светодиодами. Чем выше освещенность, тем больше светодиодов загорается. Резисторы на 1 кОм ограничивают ток через светодиоды, предотвращая их повреждение.

16. Фотосенсорный зуммер

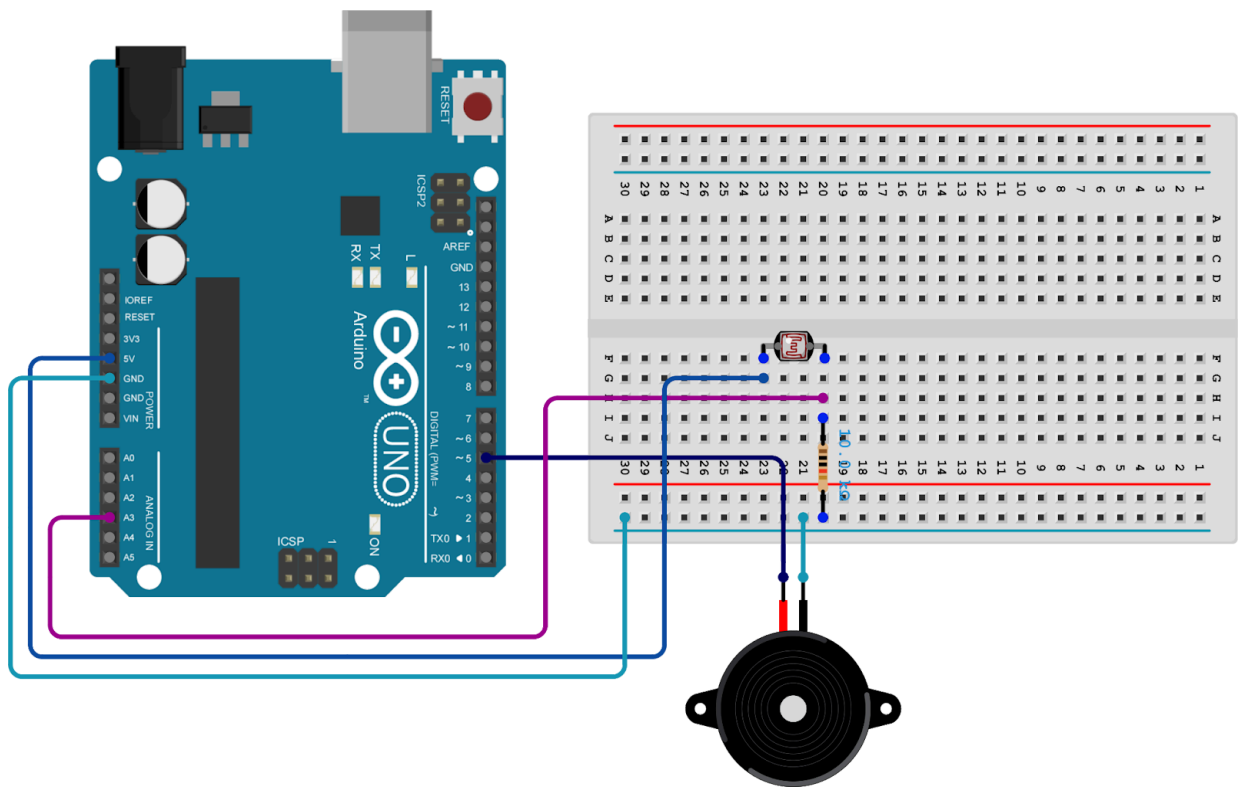
Создадим схему, которая использует фотосенсор (LDR) для управления частотой звука пьезоизлучателя, подключенного к Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Фотосенсор (LDR)
- Резистор на 10 кОм
- Пьезоизлучатель (зуммер)
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите один вывод фотосенсора к 5V на Arduino UNO.
2. Подключите другой вывод фотосенсора к аналоговому входу A3 на Arduino UNO.
3. Подключите один вывод резистора на 10 кОм к аналоговому входу A3 на Arduino UNO.
4. Подключите другой вывод резистора к GND на Arduino UNO.
5. Подключите один вывод пьезоизлучателя к цифровому выводу D5 на Arduino UNO.
6. Подключите другой вывод пьезоизлучателя к GND на Arduino UNO через резистор на 10 кОм.



Код программы:

```
// Определение номеров контактов для зуммера и фоторезистора
#define buzzerPin 5    // Номер контакта зуммера
#define photoPin A3    // Номер контакта фоторезистора

// Переменные для хранения значений
int aVal;              // Значение на фоторезисторе
int sMax=1023;         // Максимальное значение фоторезистора (10-битный АЦП Arduino)
int buzzerFreq;        // Частота звука
const long BUZZ_FREQ_MAX = 2500; // Задаем максимальную частоту излучения

void setup() {
    pinMode(buzzerPin, OUTPUT); // Установка контакта зуммера как выход
    Serial.begin(9600);         // Инициализация последовательного порта для отладки
}

void loop() {
    aVal = analogRead(photoPin); // Считываем аналоговое значение с фоторезистора
    Serial.print("aVal: ");
    Serial.println(aVal);        // Выводим значение в монитор порта
    buzzerFreq = (aVal * BUZZ_FREQ_MAX) / sMax; // Расчет частоты для зуммера
    buzz(buzzerPin, buzzerFreq, 10); // Генерация звука на зуммере
}
```

```
// Функция для генерации звука заданной частоты и продолжительности
void buzz(int targetPin, long frequency, long length) {
    long delayValue = 1000000 / frequency / 2; // Расчет времени задержки для
    полуволны
    long numCycles = frequency * length / 1000; // Расчет количества циклов звука

    // Генерация звукового сигнала
    for (long i = 0; i < numCycles; i++) {
        digitalWrite(targetPin, HIGH); // Включение пина
        delayMicroseconds(delayValue); // Задержка для создания полуволны
        digitalWrite(targetPin, LOW); // Выключение пина
        delayMicroseconds(delayValue); // Задержка для создания полуволны
    }
}
```

Принцип работы схемы:

Схема использует фотосенсор для определения уровня освещенности. Сопротивление фотосенсора изменяется в зависимости от интенсивности света, падающего на него. Это изменение сопротивления приводит к изменению напряжения на аналоговом входе A3 Arduino UNO, которое считывается и преобразуется в цифровое значение (aVal). Значение aVal используется для расчета частоты звука, которую должен излучать пьезоизлучатель. Чем выше значение aVal (чем сильнее освещенность), тем выше частота звука. Пьезоизлучатель подключен к цифровому выводу D5, который управляется функцией buzz для генерации звука заданной частоты.

17. Управление сервоприводом с помощью Arduino UNO

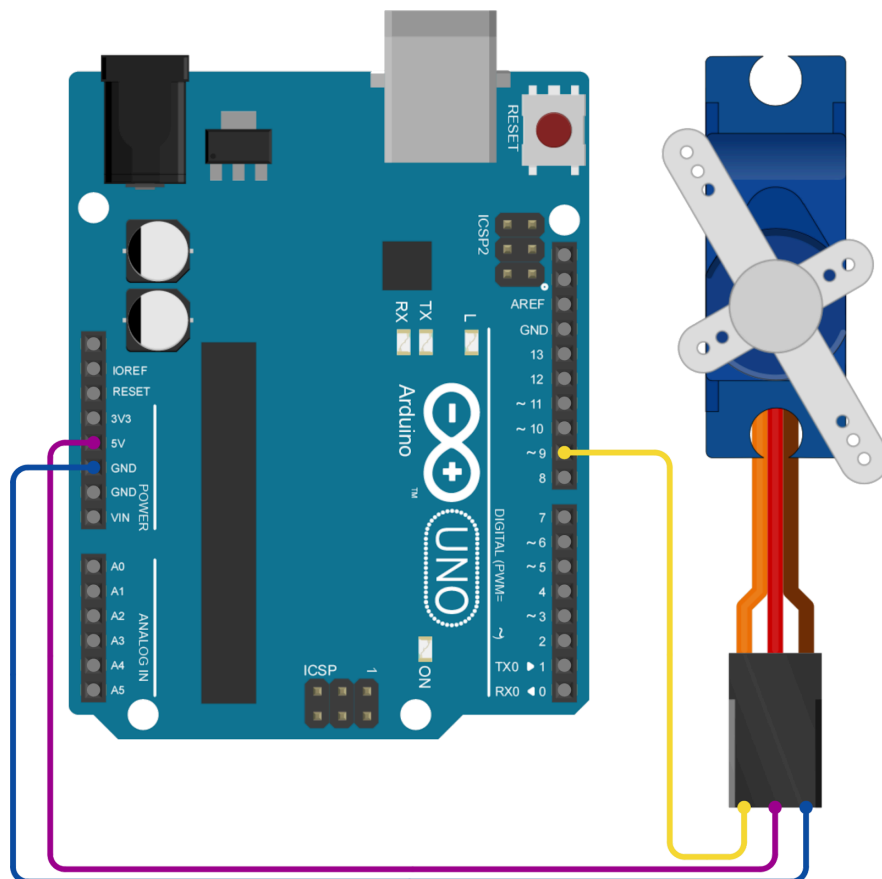
Создадим схему для демонстрации работы с ШИМ-выходами Arduino UNO, управляя углом поворота сервопривода.

Необходимые компоненты:

- Arduino UNO
- Сервопривод Tower Pro SG90

Инструкция по сборке схемы:

1. Подключите сигнальный провод сервопривода к цифровому выводу D9 на Arduino UNO.
2. Подключите питающий провод сервопривода к выводу 5V на Arduino UNO.
3. Подключите заземляющий провод сервопривода к одному из заземляющих (GND) выводов на Arduino UNO.



Код программы:

```
#include <Servo.h>
```

```

Servo myservo;  // создаем объект сервопривода

int pos = 0;    // переменная для хранения позиции сервопривода

void setup() {
  myservo.attach(9); // прикрепляем сервопривод к выводу D9
}

void loop() {
  for (pos = 0; pos <= 180; pos += 1) { // перемещаем сервопривод от 0 до 180
градусов
    myservo.write(pos);                // устанавливаем позицию сервопривода
    delay(15);                         // ожидаем 15 мс для достижения позиции
  }
  for (pos = 180; pos >= 0; pos -= 1) { // перемещаем сервопривод от 180 до 0
градусов
    myservo.write(pos);                // устанавливаем позицию сервопривода
    delay(15);                         // ожидаем 15 мс для достижения позиции
  }
}

```

Принцип работы схемы:

Схема управляет сервоприводом Tower Pro SG90 с помощью микроконтроллера Arduino UNO. ШИМ-сигнал, генерируемый на выводе D9, управляет углом поворота сервопривода.

Вот как это работает:

- Инициализация. При запуске Arduino выполняется функция `setup()`, где сервопривод прикрепляется к выводу D9 с помощью метода `myservo.attach(9);`.
- Управление сервоприводом. В функции `loop()` микроконтроллер последовательно изменяет угол поворота сервопривода от 0 до 180 градусов и обратно. Команда `myservo.write(pos);` отправляет ШИМ-сигнал на сервопривод, указывая ему занять заданную позицию.
- Задержка. Команда `delay(15);` обеспечивает паузу, необходимую сервоприводу для перемещения в новую позицию.

Таким образом, схема демонстрирует использование ШИМ-сигналов для управления сервоприводами и позволяет практиковаться в программировании микроконтроллеров для реализации точного управления механическими устройствами.

18. Управление сервоприводом с помощью потенциометра

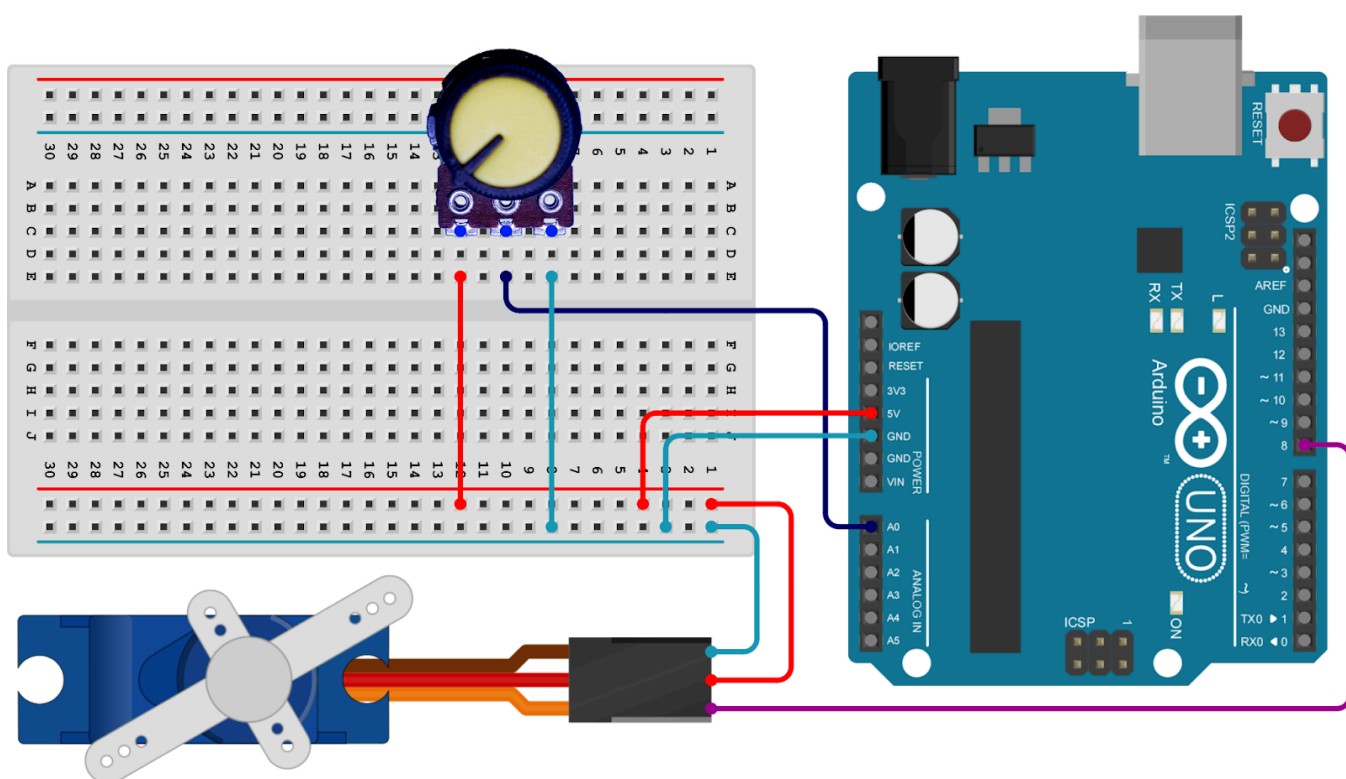
Создадим схему, позволяющую управлять положением сервопривода с помощью потенциометра, используя плату Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Сервопривод Tower Pro SG90
- Потенциометр
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите центральный вывод потенциометра к аналоговому входу A0 на Arduino UNO.
2. Подключите один из крайних выводов потенциометра к 5V, а другой к GND на Arduino UNO.
3. Подключите сигнальный провод сервопривода к цифровому выводу D8 на Arduino UNO.
4. Подключите питание (+5V) и землю (GND) сервопривода к соответствующим выводам на Arduino UNO.



Код программы:

```
#include <Servo.h> // Подключение библиотеки Servo

Servo servo1; // Создание объекта servo1 класса Servo
const int pinServo = 8; // Пин для подключения сервопривода
const int POT = 0; // Аналоговый вход A0 для подключения потенциометра
int valpot = 0; // Переменная для хранения значения потенциометра
int angleServo = 0; // Переменная для хранения угла поворота сервопривода

void setup() {
    servo1.attach(pinServo); // Подключение сервопривода к выводу pinServo
}

void loop() {
    valpot = analogRead(POT); // Чтение данных потенциометра
    angleServo = map(valpot, 0, 1023, 0, 180); // Масштабирование значения к
интервалу 0-180
    servo1.write(angleServo); // Поворот сервопривода на полученный угол
    delay(15); // Пауза для ожидания поворота сервопривода
}
```

Принцип работы схемы:

Схема позволяет управлять углом поворота сервопривода с помощью потенциометра. Потенциометр используется как делитель напряжения, а его выходное напряжение считывается аналоговым входом A0 микроконтроллера Arduino UNO. Программа преобразует аналоговое значение, считанное с потенциометра, в угол поворота сервопривода от 0 до 180 градусов. При повороте ручки потенциометра изменяется сопротивление, что приводит к изменению выходного напряжения и, соответственно, к изменению угла поворота сервопривода.

19. Инфракрасный приемник

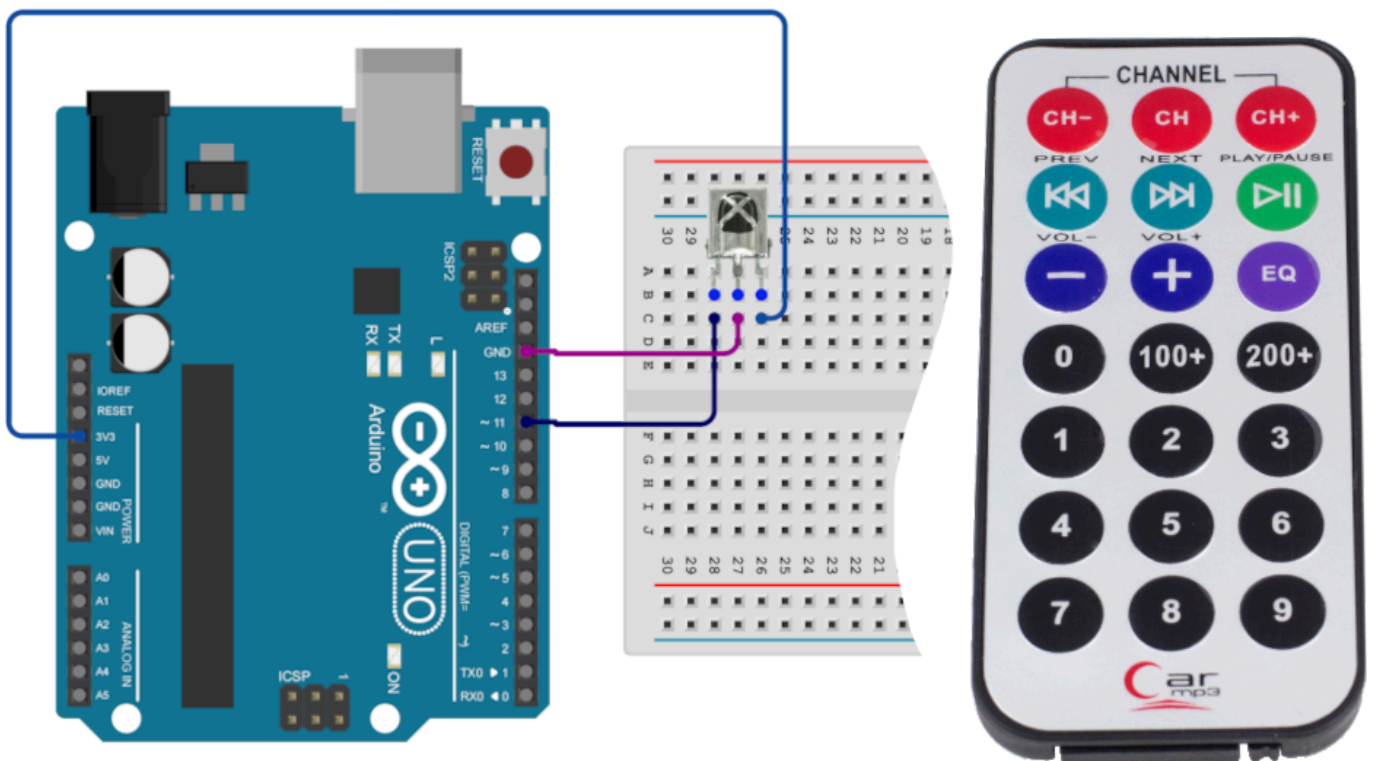
Создадим схему для приема инфракрасных сигналов с помощью Arduino UNO и инфракрасного приемника VS1838B.

Необходимые компоненты:

- Arduino UNO
- Инфракрасный приемник VS1838B
- Провода для соединения компонентов
- ИК пульт

Инструкция по сборке схемы:

1. Подключите вывод VCC инфракрасного приемника VS1838B к выводу 3.3V на Arduino UNO.
2. Подключите вывод GND инфракрасного приемника к выводу GND на Arduino UNO.
3. Подключите вывод OUT инфракрасного приемника к цифровому выводу D11 на Arduino UNO.



Код программы:

```
// Подключаем библиотеку для работы с инфракрасным приемником  
#include <IRremote.hpp>
```

```

// Определяем номер пина, к которому подключен инфракрасный приемник
#define IR_RECEIVE_PIN 11

void setup() {
    // Инициализируем последовательное соединение со скоростью 9600 бит/сек
    // для вывода данных на компьютер через USB
    Serial.begin(9600);

    // Инициализируем инфракрасный приемник на пине D11
    IrReceiver.begin(IR_RECEIVE_PIN);

    // Настраиваем пин 8 как выход (не используется в данном примере)
    pinMode(8, OUTPUT);
}

void loop() {
    // Проверяем, был ли принят инфракрасный сигнал
    if (IrReceiver.decode()) {
        // Если сигнал принят, выводим его команду в шестнадцатеричном формате
        Serial.println(IrReceiver.decodedIRData.command, HEX);

        // Задержка для стабилизации приема данных
        delay(200);

        // Подготавливаем приемник к приему следующего сигнала
        IrReceiver.resume();
    }
}

```

Принцип работы схемы:

Схема позволяет Arduino UNO принимать инфракрасные сигналы через приемник VS1838B. Приемник подключен к цифровому входу D11, который настроен на прием данных в коде программы. Когда инфракрасный приемник обнаруживает сигнал, он отправляет данные на Arduino, которое затем выводит полученную команду в шестнадцатеричном формате через последовательный порт. Это позволяет пользователю увидеть коды команд инфракрасного пульта и использовать их для управления различными функциями в своих проектах.

20. Управление шаговым двигателем

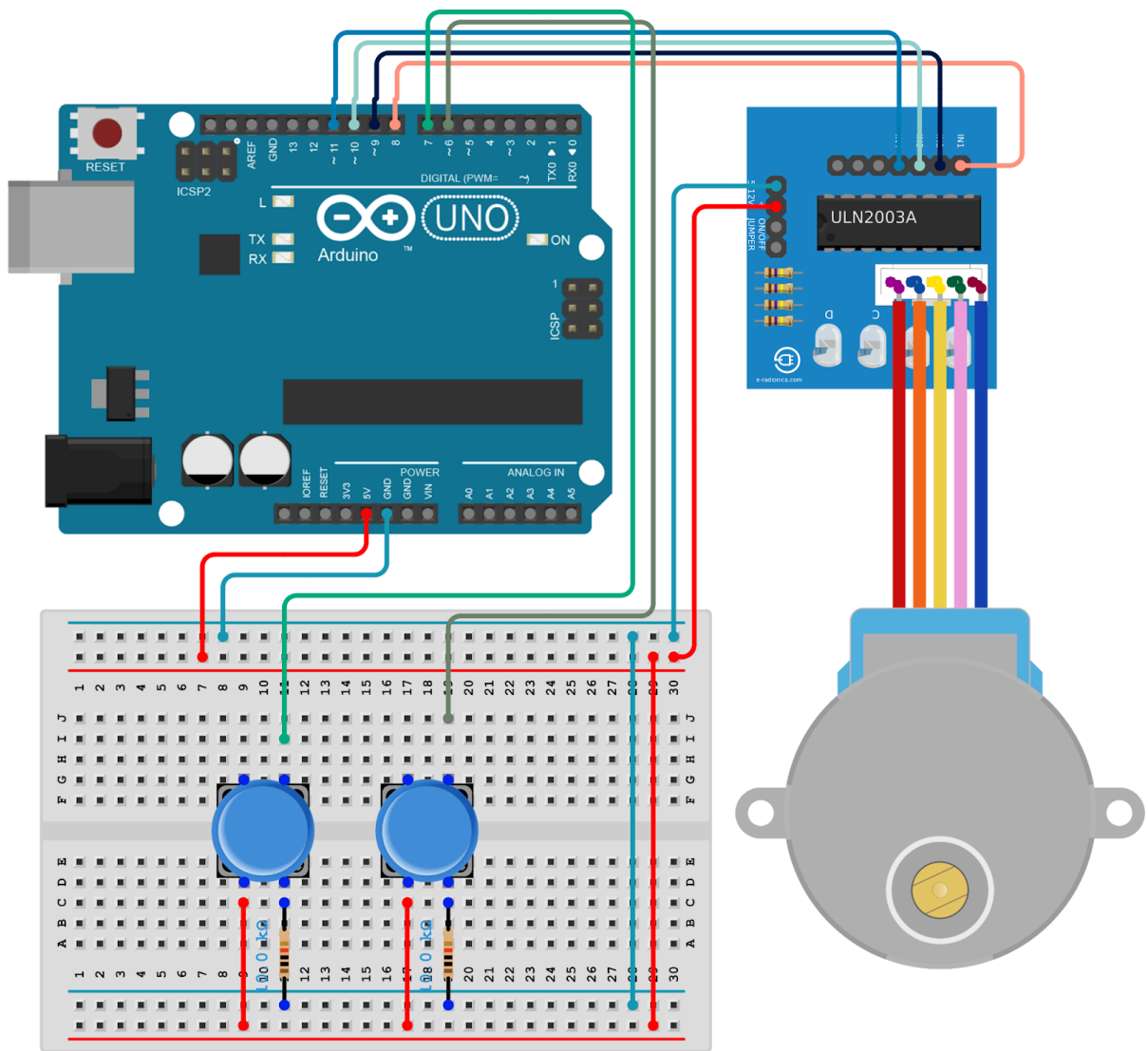
Создадим схему для управления шаговым двигателем 28BYJ-48 через драйвер ULN2003 с использованием двух кнопок для изменения направления вращения.

Необходимые компоненты:

- Arduino UNO
- Шаговый двигатель 28BYJ-48
- Драйвер ULN2003A
- 2 кнопки
- 2 резистора на 10 кОм (подтягивающие)
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите шаговый двигатель к драйверу ULN2003A, соединив соответствующие цвета проводов.
2. Подключите выводы In 1, In 2, In 3 и In 4 драйвера ULN2003A к цифровым выводам D8, D9, D10 и D11 на Arduino UNO соответственно.
3. Подключите +5V и GND драйвера к 5V и GND на Arduino UNO.
4. Подключите одну сторону каждой кнопки к цифровым выводам D6 и D7 на Arduino UNO.
5. Подключите другую сторону каждой кнопки к GND через резисторы на 10 кОм (подтягивающие резисторы).



Код программы:

```
// Подключаем библиотеку для управления шаговым двигателем через драйвер ULN2003
#include "Bonezegei_ULN2003_Stepper.h"

// Определяем количество шагов, которое должен сделать двигатель за один цикл
#define STEPS 2000

// Создаем объект Stepper для управления шаговым двигателем
// Пины 8, 9, 10, 11 Arduino UNO подключены к входам In1, In2, In3, In4
// драйвера ULN2003A
Bonezegei_ULN2003_Stepper Stepper(8, 9, 10, 11);

// Определяем константы для направления вращения двигателя
#define FORWARD 1 // Вперед
#define REVERSE 0 // Назад

// Массив с номерами пинов, к которым подключены кнопки
int pinButtons1[] = {6, 7};
```

```

// Функция setup вызывается один раз при старте программы
void setup() {
    // Инициализируем драйвер двигателя
    Stepper.begin();
    // Настраиваем пины кнопок как входы с подтяжкой к питанию
    pinMode(pinButtons1[0], INPUT_PULLUP);
    pinMode(pinButtons1[1], INPUT_PULLUP);
}

// Функция loop вызывается в бесконечном цикле после выполнения setup
void loop() {
    // Проверяем состояние первой кнопки
    if (digitalRead(pinButtons1[0]) == LOW) {
        // Если кнопка нажата (состояние LOW), вращаем двигатель вперед
        Stepper.step(FORWARD, STEPS);
    }
    // Проверяем состояние второй кнопки
    if (digitalRead(pinButtons1[1]) == LOW) {
        // Если кнопка нажата (состояние LOW), вращаем двигатель назад
        Stepper.step(REVERSE, STEPS);
    }
}

```

Принцип работы схемы:

Схема позволяет управлять шаговым двигателем с помощью Arduino UNO. Драйвер ULN2003A используется для усиления сигналов, поступающих с Arduino, и для питания шагового двигателя. Кнопки подключены к цифровым входам с подтягивающими резисторами, что позволяет считывать их состояние без помех.

При нажатии на первую кнопку двигатель вращается в одну сторону, при нажатии на вторую кнопку – в другую. Резисторы обеспечивают надежное считывание состояния кнопок, предотвращая ложные срабатывания из-за "дребезга" контактов или внешних помех.

Таким образом, данная схема демонстрирует управление шаговым двигателем и работу с цифровыми входами/выходами на практике, а также позволяет изучить основы работы с шаговыми двигателями и драйверами для них.

21. Измерение температуры

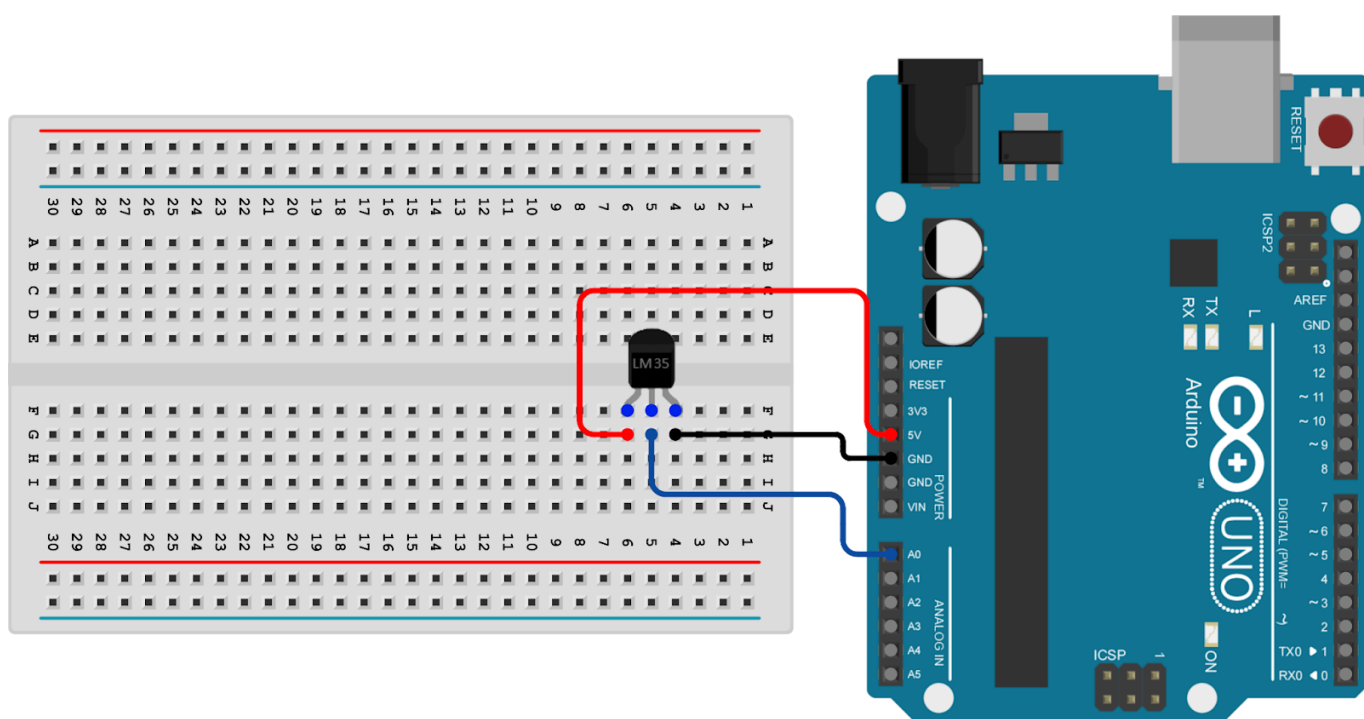
Создадим схему для измерения температуры окружающей среды с помощью датчика температуры LM35 и микроконтроллера Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Датчик температуры LM35
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите вывод GND датчика LM35 к одному из заземляющих (GND) выводов на Arduino UNO.
2. Подключите вывод Vout датчика LM35 к аналоговому входу A0 на Arduino UNO.
3. Подключите вывод Vin датчика LM35 к выводу 5V на Arduino UNO.



Код программы:

```
int raw = 0;
float temp = 0;

void setup() {
```

```

    Serial.begin(9600);
    pinMode(A0, INPUT); // Настраиваем A0 как аналоговый вход
}

void loop() {
    raw = analogRead(A0); // Читаем аналоговое значение с датчика температуры
    // Преобразуем аналоговое значение в напряжение и затем в температуру в
    градусах Цельсия
    temp = (raw * 5.0) / 1024.0 * 100.0; // LM35 выдает 10 мВ/°C, нет
    необходимости вычитать 0.5V
    Serial.println(temp); // Выводим температуру в серийный порт
    delay(1000); // Задержка в 1 секунду
}

```

Принцип работы схемы:

Датчик температуры LM35 генерирует аналоговый сигнал, который пропорционален температуре окружающей среды. Этот сигнал подается на аналоговый вход A0 микроконтроллера Arduino UNO. В функции setup() настраивается серийный порт для вывода данных и аналоговый вход A0. В основном цикле loop() происходит чтение аналогового сигнала с помощью функции analogRead(A0), преобразование его в напряжение, а затем в температуру по формуле, указанной в комментарии к коду. Полученное значение температуры выводится в серийный порт каждую секунду.

Датчик LM35 имеет линейную зависимость между выходным напряжением и температурой, где 10 мВ соответствуют 1°C. Поскольку Arduino UNO работает с опорным напряжением 5 В и 10-битным АЦП, каждый бит соответствует $5 \text{ В} / 1024 \approx 4.88 \text{ мВ}$. Формула преобразования учитывает эти значения для корректного расчета температуры в градусах Цельсия.

22. Измерение температуры и влажности с помощью DHT11

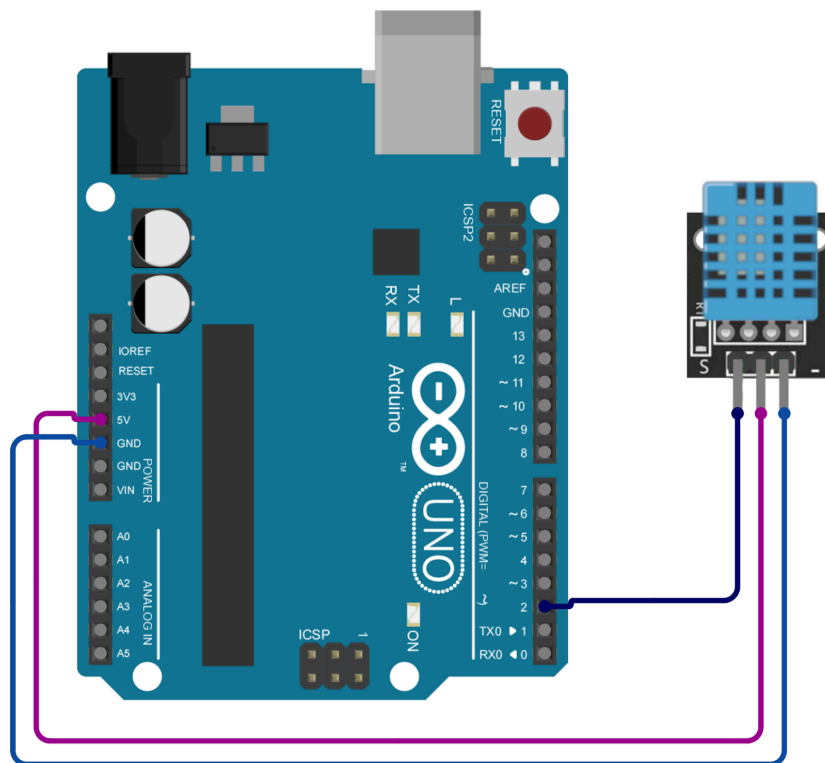
Создадим схему для считывания показаний температуры и влажности воздуха с помощью датчика DHT11 и микроконтроллера Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- Датчик температуры и влажности DHT11
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите вывод Vcc датчика DHT11 к выводу 5V на Arduino UNO.
2. Подключите вывод GND датчика DHT11 к выводу GND на Arduino UNO.
3. Подключите сигнальный вывод S датчика DHT11 к цифровому выводу D2 на Arduino UNO.



Код программы:

```
#include <DHT.h>

// Инициализация DHT11
#define DHTPIN 2      // Пин, к которому подключен датчик DHT11
#define DHTTYPE DHT11 // Используемый тип датчика DHT11
```

```

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600); // Инициализация серийного порта
  dht.begin();        // Запуск датчика DHT11
}

void loop() {
  // Чтение влажности и температуры
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  // Проверка, удалось ли считать показания с датчика
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Не удалось считать данные с датчика DHT11");
    return;
  }

  // Вывод показаний влажности и температуры в консоль
  Serial.print("Влажность: ");
  Serial.print(humidity);
  Serial.print("%\t");
  Serial.print("Температура: ");
  Serial.print(temperature);
  Serial.println("°C");

  // Задержка перед следующим замером
  delay(2000);
}

```

Принцип работы схемы:

Схема использует датчик DHT11 для измерения влажности и температуры воздуха. Arduino UNO считывает данные с датчика и выводит их в серийный порт, позволяя наблюдать показания в реальном времени через Serial Monitor.

1. Инициализация: При запуске Arduino выполняется функция `setup()`, где инициализируется серийный порт и датчик DHT11.
2. Считывание данных: В функции `loop()` происходит считывание показаний влажности и температуры.
3. Проверка данных: Программа проверяет, не являются ли считанные данные ошибочными (NaN - Not a Number).
4. Вывод данных: Если данные корректны, они выводятся в серийный порт с указанием влажности в процентах и температуры в градусах Цельсия.
5. Задержка: Между измерениями программа делает паузу в 2 секунды.

Датчик DHT11 подключается к Arduino UNO без дополнительных компонентов, так как он содержит внутренние резисторы для подтяжки. Это делает схему простой и удобной для начинающих пользователей.

23. Считыватель RFID-карт

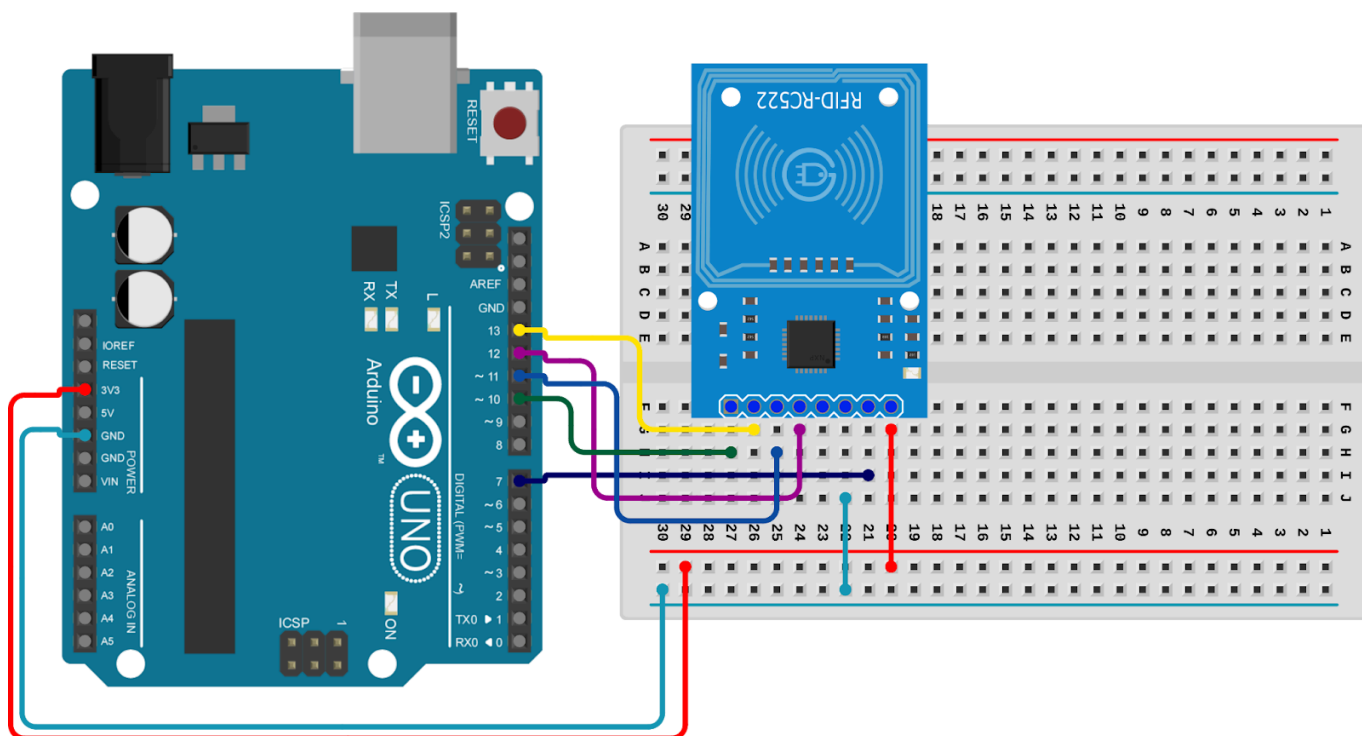
Создадим схему для демонстрации работы с RFID-технологией, используя модуль RFID-RC522 и микроконтроллер Arduino UNO для считывания идентификаторов RFID-карт или меток.

Необходимые компоненты:

- Arduino UNO
- Модуль RFID-RC522
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите модуль RFID-RC522 к Arduino UNO с помощью проводов, следуя этим соединениям:
 - 3.3V модуля к 3.3V Arduino UNO
 - GND модуля к GND Arduino UNO
 - RST модуля к цифровому выводу D7 Arduino UNO
 - MISO модуля к цифровому выводу D12 Arduino UNO
 - MOSI модуля к цифровому выводу D11 Arduino UNO
 - SCK модуля к цифровому выводу D13 Arduino UNO
 - SDA модуля к цифровому выводу D10 Arduino UNO



Код программы:

```
// Подключение библиотек
#include <SPI.h>
#include <MFRC522.h>
// Константы подключения контактов SS и RST
#define RST_PIN 7 // Внимание: здесь должен быть D7 вместо 9
#define SS_PIN 10
// Инициализация MFRC522
MFRC522 mfrc522(SS_PIN, RST_PIN); // Создание экземпляра MFRC522

void setup() {
    Serial.begin(9600); // Инициализация последовательного порта
    SPI.begin();        // Инициализация SPI
    mfrc522.PCD_Init(); // Инициализация MFRC522
}

void loop() {
    // Проверка наличия новой RFID-карты
    if (!mfrc522.PICC_IsNewCardPresent())
        return;
    // Чтение информации с RFID-карты
    if (!mfrc522.PICC_ReadCardSerial())
        return;
    // Вывод UID карты
    Serial.print(F("Card UID:"));
    for (byte i = 0; i < mfrc522.uid.size; i++) {
        Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " ");
        Serial.print(mfrc522.uid.uidByte[i], HEX);
    }
    Serial.println();
    // Вывод типа карты
    Serial.print(F("PICC type: "));
    MFRC522::PICC_Type piccType = mfrc522.PICC_GetType(mfrc522.uid.sak);
    Serial.println(mfrc522.PICC_GetTypeName(piccType));
    // Задержка перед следующим считыванием
    delay(2000);
}
```

Принцип работы схемы:

Схема позволяет считывать UID (уникальный идентификатор) и тип RFID-карт или меток с помощью модуля RFID-RC522 и Arduino UNO. Когда RFID-карта или метка приближается к модулю, он активируется и передает данные на Arduino. Программа, загруженная в Arduino, обрабатывает эти данные и выводит UID и тип карты в

последовательный порт, где их можно просмотреть с помощью монитора последовательного порта в среде разработки Arduino IDE.

Важно отметить, что в коде программы используется определение RST_PIN как 9, но в соответствии с подключением, RST_PIN должен быть изменен на 7, чтобы соответствовать фактическому подключению к выводу D7 Arduino UNO.

24. Система контроля доступа на базе RFID

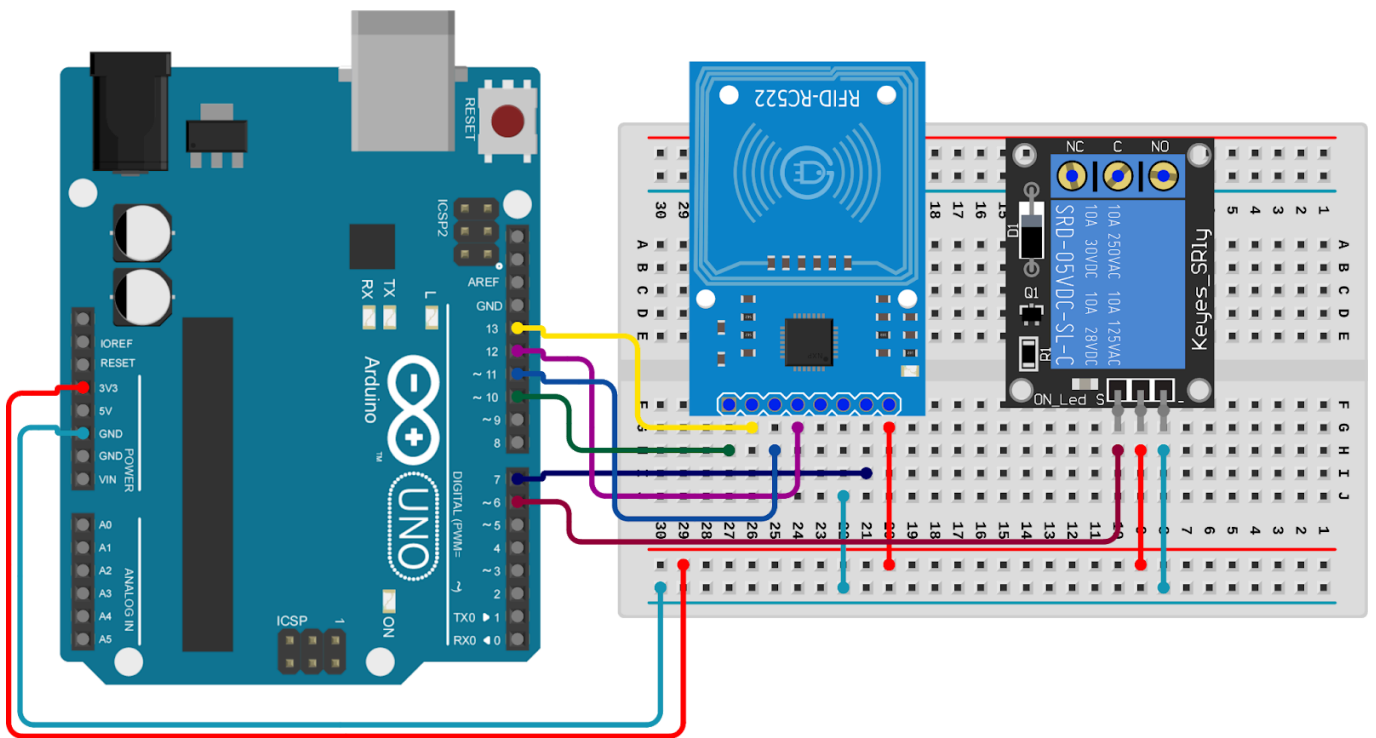
Создадим схему для демонстрации работы с RFID-технологией, используя модуль RFID-RC522 и микроконтроллер Arduino UNO для контроля доступа с возможностью управления реле.

Необходимые компоненты:

- Arduino UNO
- Модуль RFID-RC522
- 1-канальное реле (5V 10A)
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите модуль RFID-RC522 к Arduino UNO с помощью проводов, следуя этим соединениям:
 - 3.3V модуля к 3.3V Arduino UNO
 - GND модуля к GND Arduino UNO
 - RST модуля к цифровому выводу D7 Arduino UNO
 - MISO модуля к цифровому выводу D12 Arduino UNO
 - MOSI модуля к цифровому выводу D11 Arduino UNO
 - SCK модуля к цифровому выводу D13 Arduino UNO
 - SDA модуля к цифровому выводу D10 Arduino UNO
2. Подключите реле к Arduino UNO:
 - Вход "signal" реле к цифровому выводу D6 Arduino UNO (в коде используется D7, что не соответствует подключению)
 - Вход "power" реле к 5V Arduino UNO
 - Вход "ground" реле к GND Arduino UNO



Код программы:

```
#include <SPI.h>
#include <MFRC522.h>

#define RELAY_PIN 6 // Вывод для управления реле
#define RELAY_ON 0 // Уровень для включения реле
#define RELAY_OFF 1 // Уровень для выключения реле

#define RST_PIN 7 // Вывод RST для RFID-модуля
#define SS_PIN 10 // Вывод SS для RFID-модуля
MFRC522 mfrc522(SS_PIN, RST_PIN); // Создание экземпляра MFRC522

void setup() {
    Serial.begin(9600); // Инициализация последовательного порта
    SPI.begin(); // Инициализация SPI
    mfrc522.PCD_Init(); // Инициализация MFRC522
    pinMode(RELAY_PIN, OUTPUT); // Настройка вывода реле как выход
    digitalWrite(RELAY_PIN, RELAY_OFF); // Выключение реле
}

void loop() {
    // Проверка наличия новой RFID-карты
    if (mfrc522.PICC_IsNewCardPresent() && mfrc522.PICC_ReadCardSerial()) {
        // Сравнение UID карты с разрешенными значениями
        if (compare_uid(mfrc522.uid.uidByte, mfrc522.uid.size)) {
            digitalWrite(RELAY_PIN, RELAY_ON); // Включение реле
        }
    }
}
```



```

        delay(4000); // Время активации реле
        digitalWrite(RELAY_PIN, RELAY_OFF); // Выключение реле
    }
}

// Функция сравнения UID
boolean compare_uid(byte *buffer, byte bufferSize) {
    // Здесь должен быть код функции сравнения
}

```

Принцип работы схемы:

Схема позволяет считывать UID RFID-карт или меток и, при обнаружении разрешенного UID, активировать реле на определенное время. Это может использоваться для управления доступом, например, открытием дверного замка.

Инициализация. При запуске Arduino UNO выполняется функция `setup()`, где происходит инициализация последовательного порта, SPI, модуля RFID и реле.

Считывание RFID-карты. В функции `loop()` Arduino ожидает приближения RFID-карты к модулю RFID-RC522. При успешном считывании UID карты он сравнивается с разрешенными значениями.

Управление реле. Если UID карты совпадает с одним из разрешенных, реле активируется на 4 секунды, что может использоваться для открытия замка, после чего реле выключается.

Таким образом, схема демонстрирует использование RFID-технологии для контроля доступа и управления реле, а также позволяет практиковаться в программировании микроконтроллеров и работе с электронными компонентами.

25. Индикатор на ЖК-дисплее

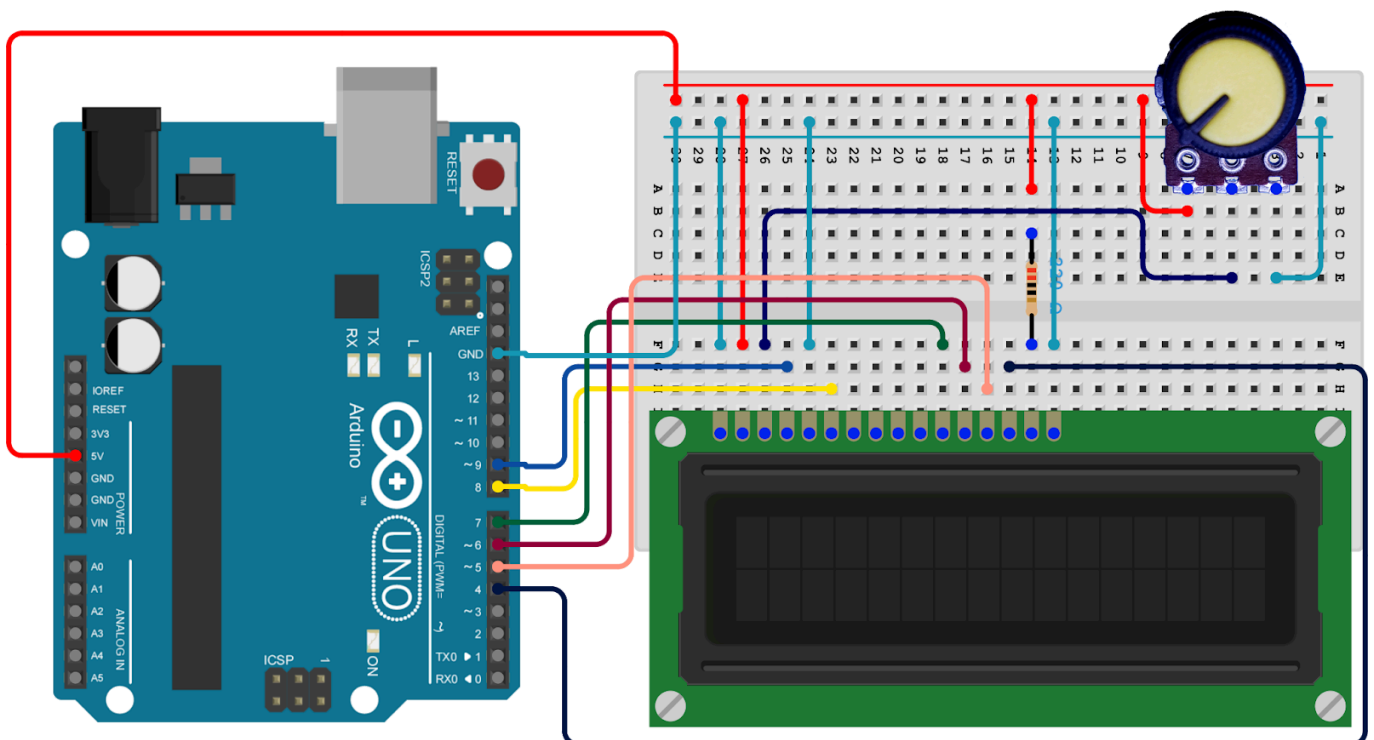
Создадим схему для отображения текстовой информации на ЖК-дисплее с использованием микроконтроллера Arduino UNO.

Необходимые компоненты:

- Arduino UNO
- ЖК-дисплей на 16 символов в 2 строки
- Потенциометр для регулировки контрастности
- Резистор на 220 Ом для подсветки
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите выводы VSS, K и R_W ЖК-дисплея к земле (GND) Arduino UNO.
2. Подключите вывод VDD ЖК-дисплея к 5V Arduino UNO через резистор на 220 Ом.
3. Подключите вывод VO ЖК-дисплея к выходу потенциометра для регулировки контрастности.
4. Подключите выводы RS, E, DB4, DB5, DB6 и DB7 ЖК-дисплея к цифровым выводам D9, D8, D7, D6, D5 и D4 Arduino UNO соответственно.
5. Подключите вывод A ЖК-дисплея к 5V Arduino UNO через резистор на 220 Ом для подсветки.



Код программы:

```
#include <LiquidCrystal.h>

// Инициализация объекта LiquidCrystal:
// RS, E, DB4, DB5, DB6, DB7
LiquidCrystal lcd(9, 8, 7, 6, 5, 4);

void setup() {
  // Установка количества столбцов и строк ЖК-дисплея:
  lcd.begin(16, 2);

  // Текст для вывода
  String line1 = "Hello!!!";
  String line2 = "Sinardcom";

  // Вычисление позиции для центрирования текста
  int position1 = (16 - line1.length()) / 2;
  int position2 = (16 - line2.length()) / 2;

  // Вывод текста на первой строке по центру:
  lcd.setCursor(position1, 0);
  lcd.print(line1);

  // Вывод текста на второй строке по центру:
  lcd.setCursor(position2, 1);
  lcd.print(line2);
}

void loop() {
  // Здесь ничего не происходит, текст статичен
}
```

Принцип работы схемы:

Схема позволяет отображать текстовую информацию на ЖК-дисплее. Arduino UNO управляет ЖК-дисплеем через библиотеку LiquidCrystal, позволяя выводить текст и управлять позиционированием символов. Потенциометр используется для регулировки контрастности дисплея, а резистор на 220 Ом ограничивает ток для подсветки, предотвращая её перегорание. После инициализации и настройки в функции setup(), ЖК-дисплей отображает заданный текст. Функция loop() пуста, так как текст на дисплее статичен и не требует обновления.

26. Подключение ЖК-дисплея i2c

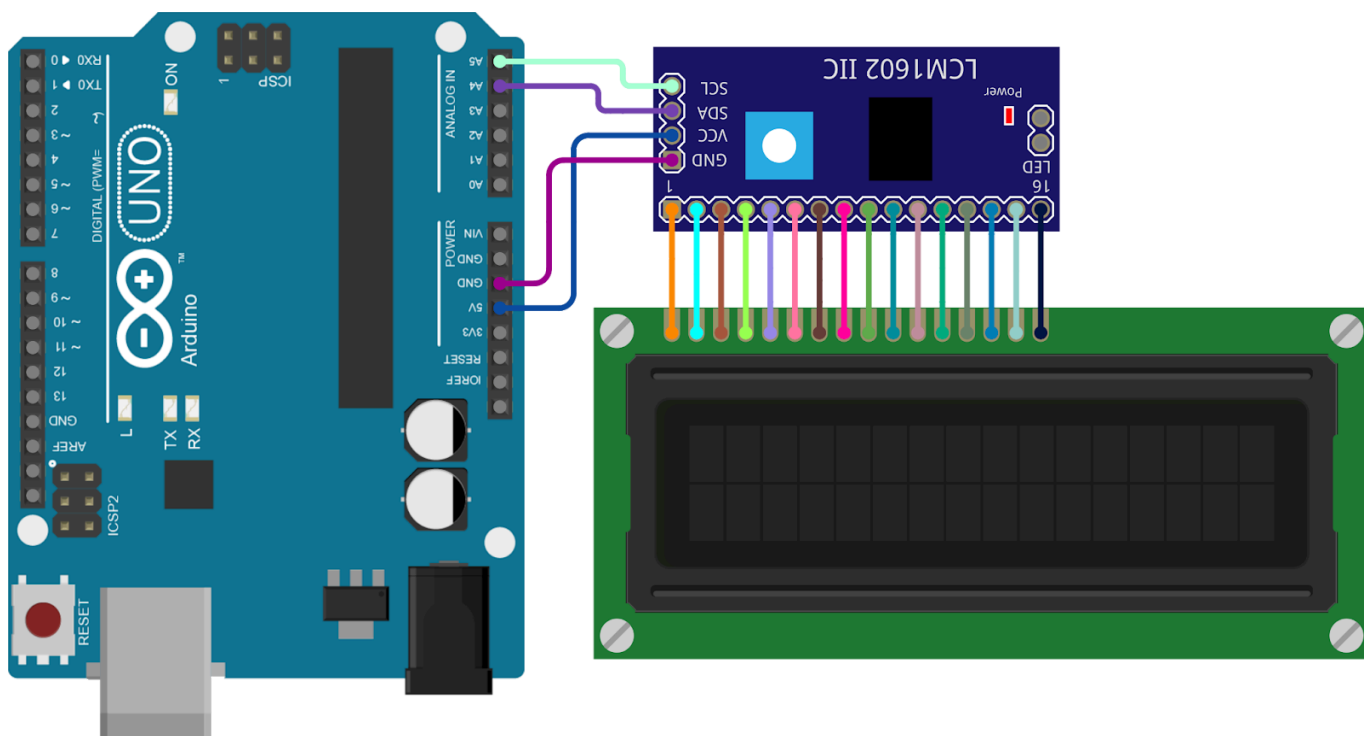
Создадим интерактивную схему, демонстрирующую использование Arduino UNO для управления ЖК-дисплеем с I2C-интерфейсом, на котором отображается статический текст "Hello!!!" по центру первой строки и прокручиваемая строка "Sinardcom" по второй строке.

Необходимые компоненты:

- Arduino UNO
- ЖК-дисплей с I2C-интерфейсом (LCM1602 IIC)
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите GND (землю) Arduino к GND ЖК-дисплея.
2. Подключите 5V Arduino к VCC ЖК-дисплея для обеспечения питания.
3. Подключите вывод A4 (SDA) Arduino к SDA ЖК-дисплея для передачи данных.
4. Подключите вывод A5 (SCL) Arduino к SCL ЖК-дисплея для синхронизации данных.



Код программы:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2); // Адрес 0x27 для I2C

void setup() {
    lcd.init();                // Инициализация ЖК-дисплея
    lcd.backlight();           // Включение подсветки
    lcd.setCursor(4, 0);       // Установка курсора в позицию (4, 0)
    lcd.print("Hello!!!");     // Вывод "Hello!!!" по центру первой
строки
}

void loop() {
    const char *text = "Sinardcom";
    int textLength = strlen(text);
    int maxPosition = 16 - textLength; // Максимальное смещение текста
    for (int position = 0; position <= maxPosition; position++) {
        lcd.setCursor(position, 1);    // Установка курсора в позицию (position,
1)
        lcd.print(text);               // Вывод "Sinardcom"
        delay(300);                   // Задержка для плавности движения
        lcd.clear();                  // Очистка экрана для следующего кадра
        lcd.setCursor(4, 0);          // Вывод "Hello!!!" по центру первой
строки
        lcd.print("Hello!!!");
    }
    for (int position = maxPosition; position >= 0; position--) {
        lcd.setCursor(position, 1);    // Установка курсора в позицию (position,
1)
        lcd.print(text);               // Вывод "Sinardcom"
        delay(300);                   // Задержка для плавности движения
        lcd.clear();                  // Очистка экрана для следующего кадра
        lcd.setCursor(4, 0);          // Вывод "Hello!!!" по центру первой
строки
        lcd.print("Hello!!!");
    }
}
```

Принцип работы схемы:

Схема использует I2C-интерфейс для управления ЖК-дисплеем. Arduino UNO отправляет команды и данные на ЖК-дисплей через два провода: SDA для данных и SCL для синхронизации.

1. Инициализация. При запуске Arduino выполняется функция `setup()`, где происходит инициализация ЖК-дисплея, включение подсветки и вывод статического текста "Hello!!!" по центру первой строки.
2. Прокрутка текста. В функции `loop()` реализован алгоритм плавного перемещения текста "Sinardcom" по второй строке дисплея от края к краю.
3. Задержка и очистка. Между каждым шагом прокрутки текста программа вставляет задержку и очищает дисплей, чтобы текст не наслаивался друг на друга.
4. Повторение. После достижения края текст начинает движение в обратном направлении, создавая эффект бегущей строки.

Этот проект позволяет познакомиться с основами работы с ЖК-дисплеями, I2C-интерфейсом и алгоритмами прокрутки текста, а также развивает навыки программирования на Arduino.

27. Часы реального времени с Arduino и RTC DS1302

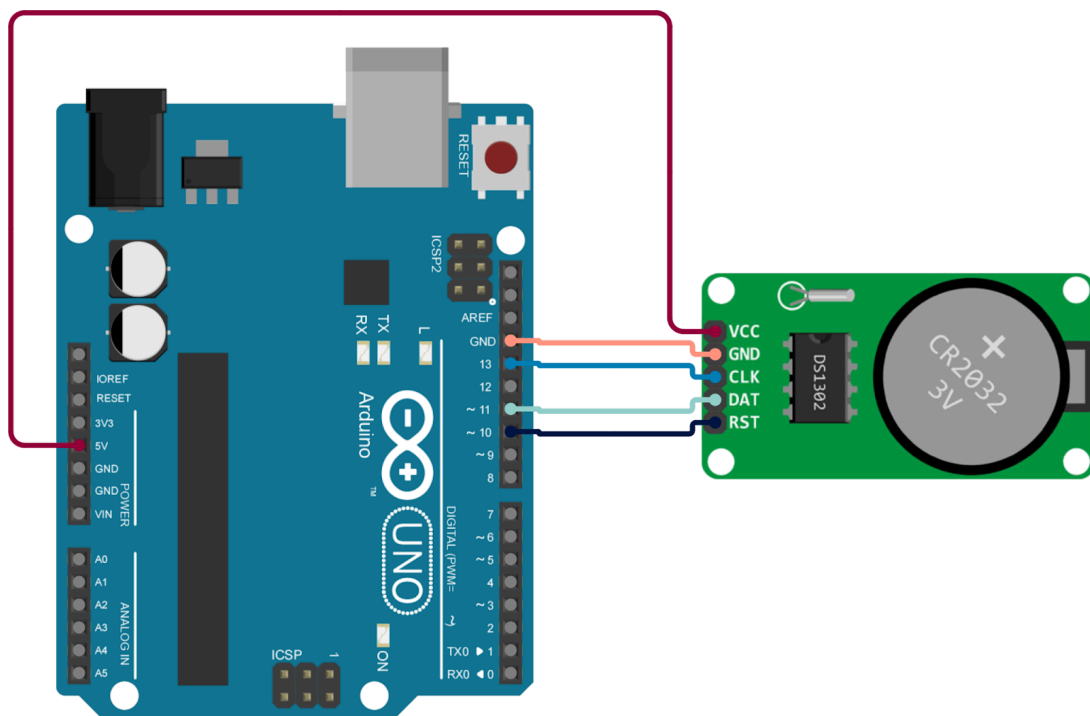
Создадим проект часов реального времени (RTC), используя плату Arduino UNO и модуль RTC DS1302 для отслеживания текущего времени и даты.

Необходимые компоненты:

- Arduino UNO
- RTC DS1302
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите вывод 5V Arduino UNO к выводу Vcc модуля RTC DS1302.
2. Подключите GND Arduino UNO к выводу GND модуля RTC DS1302.
3. Подключите вывод D10 Arduino UNO к выводу RST модуля RTC DS1302.
4. Подключите вывод D11 Arduino UNO к выводу DAT модуля RTC DS1302.
5. Подключите вывод D13 Arduino UNO к выводу CLK модуля RTC DS1302.



Код программы:

```
#include <Ds1302.h>

// Экземпляр DS1302 RTC
// Инициализация DS1302 RTC с следующими подключениями:
```

```

// Arduino D10 к RTC RST (Сброс)
// Arduino D13 к RTC CLK (Часы)
// Arduino D11 к RTC DAT (Данные)
Ds1302 rtc(10, 13, 11);

// Массив строк, представляющих дни недели
const static char* WeekDays[] =
{
    "Понедельник",
    "Вторник",
    "Среда",
    "Четверг",
    "Пятница",
    "Суббота",
    "Воскресенье"
};

// Функция setup запускается один раз при нажатии кнопки сброса или подаче
питания на плату
void setup()
{
    // Начать серийную связь со скоростью 9600 бит в секунду
    Serial.begin(9600);

    // Инициализировать RTC
    rtc.init();

    // Проверить, остановлен ли RTC, и если да, то установить время
    if (rtc.isHalted())
    {
        Serial.println("RTC остановлен. Установка времени...");

        // Установить RTC на определенную дату и время
        Ds1302::DateTime dt = {
            .year = 24, // Год (00-99)
            .month = Ds1302::MONTH_JUN, // Месяц в виде перечисления
            .day = 25, // День месяца (1-31)
            .hour = 11, // Час дня (0-23)
            .minute = 00, // Минута (0-59)
            .second = 0, // Секунда (0-59)
            .dow = Ds1302::DOW_TUE // День недели в виде перечисления
        };

        // Установить дату и время на RTC
        rtc.setDateTime(&dt);
    }
}

```



```

}

// Функция loop выполняется снова и снова бесконечно
void loop()
{
    // Получить текущее время от RTC
    Ds1302::DateTime now;
    rtc.getDateTime(&now);

    // Отслеживать последнюю обработанную секунду
    static uint8_t last_second = 0;
    if (last_second != now.second)
    {
        last_second = now.second;

        // Вывести текущую дату и время на серийный монитор
        Serial.print("20"); // Префикс для года, чтобы сделать его
// четырехзначным
        Serial.print(now.year); // Вывести год (00-99)
        Serial.print('-');
        if (now.month < 10) Serial.print('0'); // Вывести ведущий ноль для
// месяца
        Serial.print(now.month); // Вывести месяц (01-12)
        Serial.print('-');
        if (now.day < 10) Serial.print('0'); // Вывести ведущий ноль для дня
        Serial.print(now.day); // Вывести день (01-31)
        Serial.print(' ');
        Serial.print(WeekDays[now.dow - 1]); // Вывести день недели (1-7)
        Serial.print(' ');
        if (now.hour < 10) Serial.print('0'); // Вывести ведущий ноль для
// часа
        Serial.print(now.hour); // Вывести час (00-23)
        Serial.print(':');
        if (now.minute < 10) Serial.print('0'); // Вывести ведущий ноль для
// минуты
        Serial.print(now.minute); // Вывести минуту (00-59)
        Serial.print(':');
        if (now.second < 10) Serial.print('0'); // Вывести ведущий ноль для
// секунды
        Serial.print(now.second); // Вывести секунду (00-59)
        Serial.println(); // Закончить строку
    }

    // Подождать полсекунды
    delay(500);
}

```

Принцип работы схемы:

Проект использует плату Arduino UNO для управления модулем RTC DS1302, который поддерживает точное время даже при отключении питания Arduino благодаря встроенной батарее. Arduino запрашивает текущее время с модуля RTC и выводит его через серийный порт, позволяя пользователю видеть текущие дату и время.

Вот как это работает:

1. Инициализация. При запуске или перезагрузке Arduino выполняется функция `setup()`, которая инициализирует серийную связь и модуль RTC. Если RTC был остановлен, устанавливается текущее время.
2. Чтение времени. В функции `loop()` Arduino запрашивает текущее время с модуля RTC.
3. Вывод времени. Arduino выводит информацию о текущем времени и дате в серийный порт, где её можно просмотреть с помощью серийного монитора в среде разработки Arduino IDE.
4. Повторение. Функция `loop()` выполняется бесконечно, обновляя и выводя время каждые полсекунды.

Таким образом, проект демонстрирует использование модуля RTC для создания часов реального времени с Arduino, что может быть полезно для различных приложений, требующих отслеживания времени и даты.

28. Станция мониторинга температуры, влажности и времени

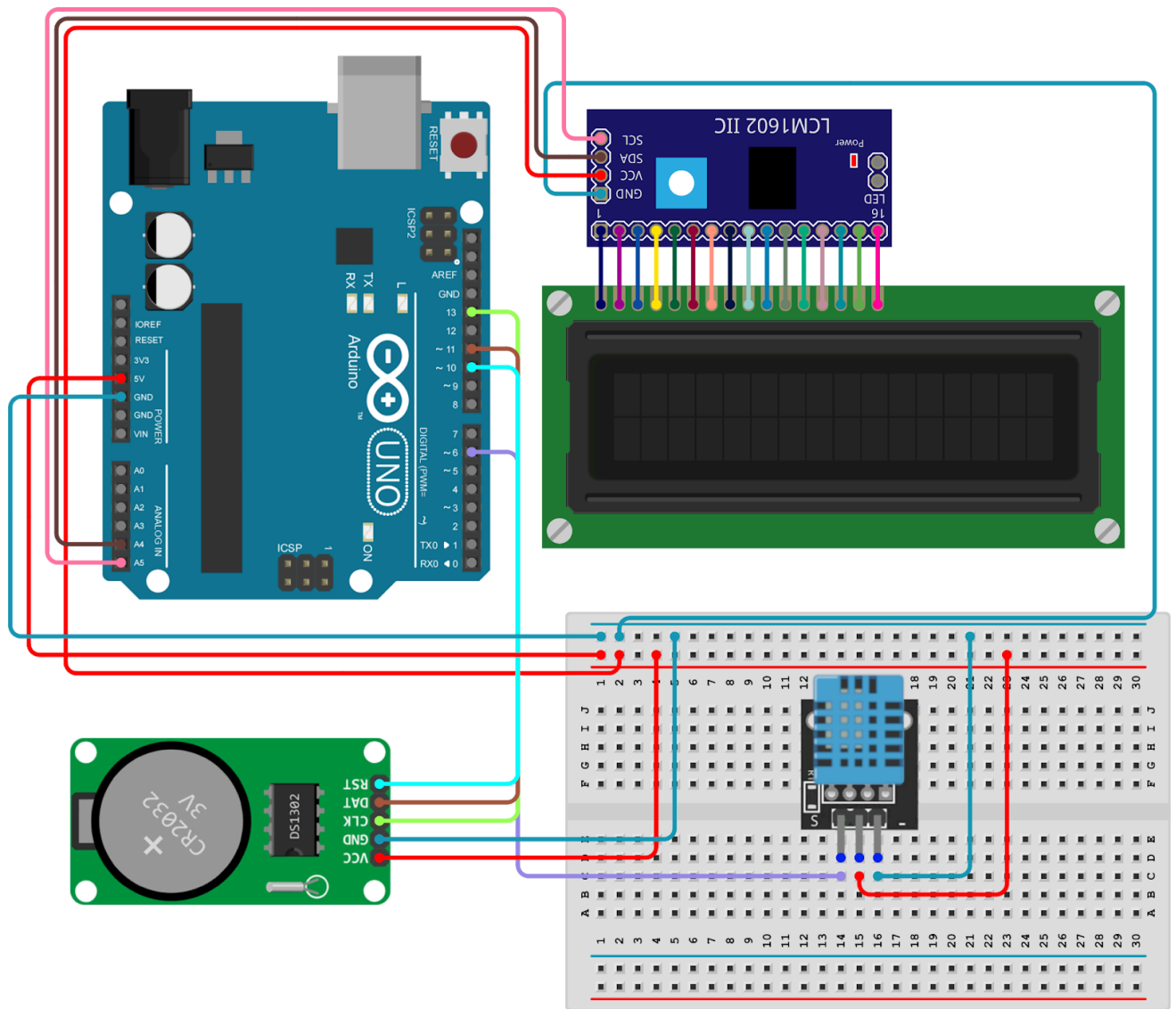
Создадим многофункциональную станцию для мониторинга температуры и влажности с отображением текущего времени и даты на LCD-дисплее.

Необходимые компоненты:

- Arduino UNO
- Датчик температуры и влажности DHT11
- Модуль реального времени (RTC) DS1302
- LCD-дисплей на 16 символов в 2 строки с интерфейсом I2C
- Провода для соединения компонентов

Инструкция по сборке схемы:

1. Подключите VCC и GND датчика DHT11 к 5V и GND на Arduino UNO соответственно.
2. Подключите DATA-вывод датчика DHT11 к цифровому выводу D6 на Arduino UNO.
3. Подключите VCC и GND модуля RTC DS1302 к 5V и GND на Arduino UNO соответственно.
4. Подключите CLK, DAT и RST выводы модуля RTC DS1302 к цифровым выводам D13, D11 и D10 на Arduino UNO соответственно.
5. Подключите VCC и GND LCD-дисплея с интерфейсом I2C к 5V и GND на Arduino UNO соответственно.
6. Подключите SDA и SCL выводы интерфейса I2C к аналоговым выводам A4 и A5 на Arduino UNO соответственно.



Код программы:

```
#include <Wire.h>
#include <DHT.h>
#include <LiquidCrystal_I2C.h>
#include <Ds1302.h>

// Инициализация DHT11
#define DHTPIN 6 // Пин, к которому подключен DHT11
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// Пины для RTC DS1302
#define RTC_RST 10 // Пин RST для DS1302
#define RTC_DAT 11 // Пин DAT для DS1302
#define RTC_CLK 13 // Пин CLK для DS1302

// Инициализация RTC DS1302
```

```

Ds1302::DateTime t;
Ds1302 rtc(RTC_RST, RTC_CLK, RTC_DAT);

// Инициализация LCD дисплея
LiquidCrystal_I2C lcd(0x27, 16, 2); // Адрес I2C и размерность дисплея

void setup() {
    // Инициализация интерфейсов
    Wire.begin();
    dht.begin();
    rtc.init();
    lcd.init(); // Инициализация дисплея
    lcd.backlight(); // Включение подсветки
    if (rtc.isHalted())
    {
        Serial.println("RTC остановлен. Установка времени...");
        // Установить RTC на определенную дату и время
        Ds1302::DateTime dt = {
            .year = 24, // Год (00-99)
            .month = Ds1302::MONTH_JUN, // Месяц в виде перечисления
            .day = 25, // День месяца (1-31)
            .hour = 11, // Час дня (0-23)
            .minute = 00, // Минута (0-59)
            .second = 0, // Секунда (0-59)
            .dow = Ds1302::DOW_TUE // День недели в виде перечисления
        };
        // Установить дату и время на RTC
        rtc.setDateTime(&dt);
    }
}

void loop() {
    // Получение текущего времени и даты
    rtc.getDateTime(&t);

    float temp = dht.readTemperature(); // Чтение температуры
    float humidity = dht.readHumidity(); // Чтение влажности

    // Формирование строки с датой и временем
    char dateTimeStr[17];
    sprintf(dateTimeStr, "%02d-%02d-%04d %02d:%02d", t.day, t.month, t.year +
2000, t.hour, t.minute);

    // Формирование строки с днем недели, температурой и влажностью
    char statusStr[17] = {0}; // Инициализация строки нулями

```

```

// Копирование дня недели в начало строки
const char* dow = dayOfWeek(t.dow);
sprintf(statusStr, "%s", dow);

// Форматирование температуры и влажности и добавление в конец строки
char tempHumStr[10];
sprintf(tempHumStr, "%02dC %02d%", (int)temp, (int)humidity);

// Выравнивание температуры и влажности справа
int spaceCount = 16 - strlen(statusStr) - strlen(tempHumStr); //
Количество пробелов для заполнения
for (int i = 0; i < spaceCount; i++) {
    strcat(statusStr, " "); // Добавление пробелов для выравнивания
}
strcat(statusStr, tempHumStr); // Добавление температуры и влажности в конец
строки

// Отображение информации на LCD
lcd.setCursor(0, 0);
lcd.print(dateTimeStr);
lcd.setCursor(0, 1);
lcd.print(statusStr);

delay(1000); // Обновление информации раз в секунду
}

// Функция для получения строки дня недели
const char* dayOfWeek(int dayIndex) {
    const char* daysOfWeek[] = { "Mon", // "Понедельник"
        "Tue", // "Вторник"
        "Wed", // "Среда"
        "Thu", // "Четверг"
        "Fri", // "Пятница"
        "Sat", // "Суббота"
        "Sun" // "Воскресенье"
    };
    return daysOfWeek[dayIndex - 1]; // Индексация дней недели начинается с 1
}

```

Принцип работы схемы:

Схема использует Arduino UNO для сбора данных с датчика DHT11 и модуля RTC DS1302, а также для управления LCD-дисплеем через интерфейс I2C. Датчик DHT11 измеряет температуру и влажность окружающей среды, а модуль RTC DS1302 предоставляет точное текущее время и дату.

1. Инициализация. При запуске Arduino UNO выполняется функция `setup()`, где инициализируются интерфейсы связи с датчиком DHT11, модулем RTC и LCD-дисплеем.
2. Считывание данных. В основном цикле `loop()` Arduino считывает текущие показания температуры и влажности с датчика DHT11 и текущее время и дату с модуля RTC DS1302.
3. Отображение информации. Сформированные данные отображаются на LCD-дисплее, где верхняя строка используется для даты и времени, а нижняя строка - для показаний температуры и влажности с выравниванием по краям.
4. Обновление данных. Информация на дисплее обновляется каждую секунду.

Роль компонентов:

- DHT11: измеряет температуру и влажность.
- RTC DS1302: поддерживает точное время и дату даже при отключении питания Arduino благодаря встроенной батарее.
- LCD-дисплей: отображает измеренные и сформированные данные для пользователя.
- Arduino UNO: управляет всеми компонентами и обрабатывает данные.

Таким образом, проект демонстрирует возможности Arduino UNO в сборе и обработке данных с различных датчиков и модулей, а также в управлении периферийными устройствами, такими как LCD-дисплей.

Приложение А. Состав набора

- Состав комплекта:
- Перемычка папа-папа: 16 шт.
- LCD-дисплей 1602A: 1 шт.
- Converter в IIC/I2C: 1 шт.
- Ультразвуковой датчик HC-SR04: 1 шт.
- Светодиодная матрица 8x8 1588bs: 1 шт.
- Джойстик двухосевой: 1 шт.
- Модуль RFID Считыватель/программатор карт с картой и брелоком: 1 шт.
- Шаговый двигатель: 1 шт.
- Arduino UNO R3: 1 шт.
- Четырехразрядный семисегментный индикатор: 1 шт.
- ИК пульт: 1 шт.
- USB кабель: 1 шт.
- Макетная плата 830 точек: 1 шт.
- Датчик уровня воды: 1 шт.
- Модуль RGB светодиода: 1шт.
- Датчик температуры и влажности DHT-11: 1 шт.
- Кнопочная матрица 4x4: 1 шт.
- Одноразрядный семисегментный индикатор: 1 шт.
- Сервопривод SG90 9г: 1шт.
- Кабель питания от кроны 9V: 1 шт.
- Перемычки папа-мама: 10 шт.
- Преобразователь интерфейса LCD в I2C для дисплеев: 1 шт.
- Кнопка тактовая: 4 шт.
- Потенциометр 10 кОм: 1 шт.
- Светодиод 5мм красный: 5 шт.
- Светодиод 5мм зеленый: 5 шт
- Светодиод 5мм желтый: 5 шт.
- Фоторезистор 5516: 3 шт.
- Микросхема sn74hc595n сдвиговой регистр: 1 шт.
- Зуммер: 2шт.
- Датчик наклона и вибрации SW-520D: 1 шт.
- ИК-приемник cho 1838: 1 шт.
- Микрофонный модуль датчика звука: 1 шт.
- Датчик температуры LM35: 1 шт.
- ИК передатчик: 1 шт.
- Резистор 1 кОм: 10 шт.
- Резистор 10 кОм: 10 шт.
- Резистор 220 Ом: 10 шт.
- Плата управления шаговыми двигателями: 1 шт.
- Модуль DS1302 часы в реальном времени с баратрейкой: 1 шт.
- 1 каналный релейный модуль плата щит для PIC AVR DSP ARM: 1 шт.
- Инструкции и проекты для Arduino UNO – 1 шт.